PERSONAL STUDY GUIDE · FROM ZERO TO FINAL IMPLEMENTATION

Volvo Battery Passport

Blockchain-Based Supply Chain Transparency

A beginner-friendly guide to everything in this project: the problem it addresses, the blockchain ideas behind it, every file we created, how the smart contract, tests, scripts and dashboard work, what has been verified, what is still pending, and how to explain and defend it in a presentation and viva.

1	5	6	10
Solidity smart contract BatteryPassport.sol	on-chain roles	lifecycle stages	passing contract tests

Educational prototype inspired by automotive battery traceability (Volvo Cars–Circulor). It is not an official Volvo Cars or Circulor product and is not connected to their systems. It runs only on a local Hardhat blockchain (chain ID 31337). All battery data is fictional. Prepared October 2026 from the actual project files in `VolvoBatteryPassport\`.

Contents

—	How to use this guide	3
Chapter 1	Understand the project before the technology	4
Chapter 2	Blockchain fundamentals from zero	7
Chapter 3	The architecture of our project	14
Chapter 4	Environment setup and project structure	18
Chapter 5	The smart contract, concept by concept	23
Chapter 6	Testing the contract	32
Chapter 7	Deployment, seeding and frontend export	36
Chapter 8	Frontend walkthrough	40
Chapter 9	MetaMask and local test accounts	45
Chapter 10	Demonstration: our actual progress	48
Chapter 11	Remaining work and completion checklist	50
Chapter 12	Security, limitations and future improvements	53
Chapter 13	Viva and presentation preparation	56
	Viva questions and answers	57
	Glossary	60
Appendix	Project progress checklist	63
Final page	One-page revision sheet	64

How to use this guide

Read the chapters in order the first time. Chapters 1–3 give you the ideas, Chapters 4–9 explain every part of the code, Chapters 10–11 tell you exactly where the project stands and what to do next, and Chapters 12–13 prepare you for questions. The final page is a one-page summary for the morning of your viva.

Most technical ideas are explained in a **concept card** with six parts: a simple definition, an analogy, how it works technically, how our project uses it, what you can say in your presentation, and a likely viva question.

Every claim about the project carries one of four labels:

Label	Meaning
VERIFIED	Implemented and confirmed by a real run: compile output, test output, script output, or your own observation in the dashboard.
NOT YET VERIFIED	The code exists (you can read it in the project), but nobody has yet run it end-to-end in the dashboard with MetaMask.
PENDING	A step that still has to be carried out to finish the project (Chapter 11).
FUTURE WORK	An idea for a production system. Not implemented in this project.

Where the facts come from

Code, function names, error messages, line numbers and commands are taken from the project files as they exist now (`contracts/` , `test/` , `scripts/` , `frontend/src/`). Results come from real command output recorded while building the project, or from the milestones you reported (for example, `DEMO-BATCH-0002` registered in block 12). Where something could not be checked, the guide says so.

CHAPTER 1

Understand the project before the technology

Before any code: what problem are we solving, who is involved, and what exactly did our prototype set out to show? If you can explain this chapter clearly, you can open any presentation confidently.

1.1 The problem: where did this battery come from?

An electric-vehicle (EV) battery is made from minerals such as **lithium, cobalt, nickel and graphite**. These minerals are dug out of the ground in one country, purified in another, turned into battery cells in a third, installed in a car in a fourth, driven for years, and finally recycled. At every step a *different organisation* holds the material and keeps its own records.

That creates three practical problems:

- **Fragmented records.** Each company has its own database. No single place shows the whole journey of one batch of material.
- **Claims are hard to check.** "This cobalt was mined responsibly", "this battery contains 16 % recycled material", "its carbon footprint is 5,600 kg CO_2e" are statements made by different parties. A customer or regulator cannot easily verify them.
- **Regulation now asks for this data.** The EU Battery Regulation (EU) 2023/1542 requires a **digital battery passport** for EV batteries placed on the EU market from February 2027. Manufacturers therefore need a reliable way to collect lifecycle data from their suppliers.

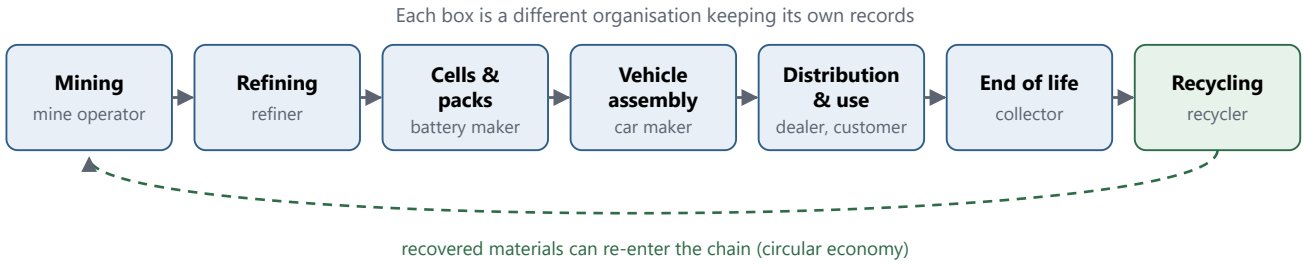


Figure 1.1 — The journey of battery material, from mine to recycler.

1.2 A worked example: following one batch

Imagine a fictional batch called `DEMO-BATCH-0001` (this is the real demo batch in our project; its data is invented):

1. A mining company extracts cobalt, lithium and nickel at "Demo Mine Site A". It records the **mine location** and **material origin**.
2. A refiner purifies the material and states that **16 % of the input was recycled**.
3. A battery manufacturer assembles cells and packs and reports a **carbon footprint of 5,600 kg CO_2e**. The battery is installed in a vehicle with VIN `DEMOVIN0000000001` and passes **quality inspection**.
4. The vehicle is shipped to a dealership.
5. Years later the battery reaches **end of life** and is collected.
6. A recycler records the **recovered materials** (lithium, cobalt, nickel).

A customer who scans a QR code on the battery should be able to see this history. In our project, every one of these six steps is a separate, signed record on a blockchain, and the customer's view is called the **public battery passport**.

1.3 Key words you must be able to define

Term	Simple meaning	In our project
Battery passport	A digital record that travels with a battery and describes its origin, materials, carbon footprint and lifecycle.	The data returned by `getPublicPassport`, shown on the dashboard with a QR code.
Supply chain	All the organisations and steps that turn raw material into a finished product and back again.	Mining → refining → manufacturing → distribution → end of life → recycling.
Transparency	The right people can *see* the information.	Anyone can read the public passport without a wallet.
Traceability	You can *follow* an item step by step back to its origin.	Each batch has a stage history and events showing who advanced it.
Accountability	Every record can be tied to *who* made it.	Every write is a transaction signed by a known account; `registeredBy` stores the supplier's address.
Immutability	Once recorded, a record cannot be quietly changed or deleted.	The contract has no function to edit or delete a stage once recorded.

1.4 Why might blockchain help?

A traditional solution would be one big central database owned by, say, the car maker. Every supplier would have to trust that owner not to change records. A **blockchain** (explained fully in Chapter 2) is a shared record book that many parties can read, where entries are added by signed transactions, and where rules are enforced by code everyone can inspect. That suits a supply chain in which participants do not fully trust each other.

> **The most important limitation, stated early**
>
> A blockchain proves **who** recorded something and that it has **not changed since**. It does **not** prove that the information was **true** when it was entered. If a supplier types a false mine location, the false value is stored permanently. Real systems add audits, certificates and independent checks. Our prototype does not do this, and you should say so.

1.5 The inspiration: Volvo Cars and Circulor

BACKGROUND, NOT PART OF OUR CODE According to public announcements, summarised in your case-study report, Volvo Cars began working with Circulor, a UK traceability company, around 2019 to trace cobalt in its EV batteries using blockchain-based technology. Volvo Cars later invested in Circulor through its Tech Fund. In 2024, Volvo announced a battery passport for the EX90 ahead of the EU rules. This guide does not verify these details independently; treat them as context from your report.

Our project is **inspired by** that idea. It is not a copy of their platform, does not use their software, and is not connected to Volvo or Circulor in any way. "Volvo" appears in the project name only because the case study inspired it.

1.6 What our prototype does, and what it does not do

It does (implemented)

- Stores battery batches in one Solidity smart contract on a local blockchain.
- Enforces five roles: Admin, Supplier, Manufacturer, Regulator, Customer.
- Enforces six lifecycle stages in strict order.
- Requires every update to be signed by the participant's own MetaMask account.
- Shows a public passport that needs no wallet, and a QR code linking to it.
- Gives Admin and Regulator a fuller view.
- Has 10 automated tests.

It does not (out of scope)

- Run on a public blockchain or a real permissioned network.
- Connect to Volvo, Circulor or any real supplier.
- Verify that submitted data is true.
- Keep data confidential: on-chain data is readable by anyone with access to the node.
- Use IoT sensors, battery-health data or audits.
- Allow corrections once a stage is recorded.

What you can say

"My project is an educational prototype of a battery passport. It records each stage of a battery batch's life on a local blockchain, lets only authorised roles record each stage in the correct order, and lets anyone view a public passport through a QR link. It shows how blockchain can provide traceability and accountability, and also where blockchain alone is not enough."

CHAPTER 2

Blockchain fundamentals from zero

Only the concepts you need to understand and defend this project. Each comes with an analogy first, then the technical idea, then how our project uses it.

2.1 The big picture in one analogy

Picture a **shared notebook** kept in a glass cabinet in a village square. Anyone can read it. To add a line you must sign it with your personal stamp, and a referee checks the rules before the line is accepted. Pages are numbered and glued together, so tearing one out is obvious. That notebook is a blockchain, the stamps are digital signatures, the referee's rulebook is a smart contract, and the glue is cryptographic hashing.

Blockchain

DEFINITION	A database made of **blocks** of records, where each block points to the previous one using a cryptographic fingerprint, so past records cannot be changed without detection.
ANALOGY	The glued, numbered notebook in the glass cabinet.
HOW IT WORKS	Nodes (computers running the blockchain software) hold a copy of the data. New transactions are grouped into a block; the block includes the hash of the previous block, forming a chain.
OUR PROJECT	We run one local blockchain with Hardhat on your laptop. It behaves like Ethereum, but only you run it.
SAY THIS	"A blockchain is an append-only shared ledger. Each block references the previous block's hash, which makes tampering detectable."
VIVA QUESTION	*What makes a blockchain tamper-evident?* Each block contains the previous block's hash; changing old data changes its hash and breaks every link after it.

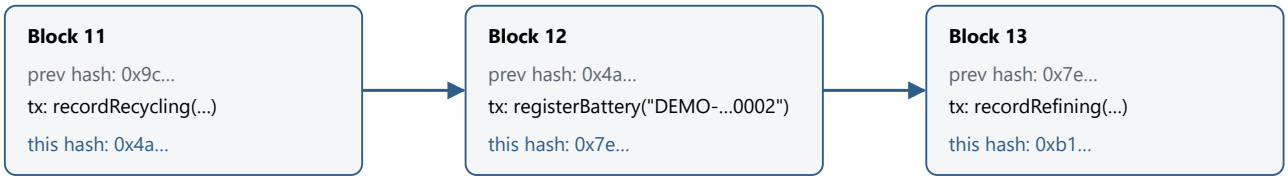


Figure 2.1 — Blocks linked by hashes. Hash values are shortened and illustrative. Block 12 really did contain the registration of DEMO-BATCH-0002; the block numbers of the other transactions shown are illustrative.

Transaction

DEFINITION	A signed request to change the blockchain's state, for example "call `registerBattery` with these values".
ANALOGY	A signed, dated form handed to the referee.

Transaction

HOW IT WORKS	It contains the sender, the target contract address, the encoded function call (*calldata*), a nonce (a counter that prevents replay), gas settings and a signature. A node checks the signature, executes it and includes it in a block. The result is a **receipt** saying success or revert.
OUR PROJECT	Every lifecycle update and role change is a transaction signed in MetaMask, for example registering `DEMO-BATCH-0002`, which was included in block 12.
SAY THIS	"Every update on the passport is a signed transaction, so it is tied to an account and recorded in a block."
VIVA QUESTION	*What is a transaction receipt?* The node's record of the result: block number, status (success or reverted), gas used and emitted events (logs).

Hash

DEFINITION	A fixed-length fingerprint computed from any data. Changing even one character of the data gives a completely different fingerprint.
ANALOGY	A fingerprint for a document.
HOW IT WORKS	Ethereum uses the Keccak-256 hash function. Hashes identify blocks and transactions (the "transaction hash" you see after confirming) and link blocks together.
OUR PROJECT	The dashboard shows each transaction's hash. Our events also store a hash: because `batchId` is declared `indexed` in the events, the log stores the Keccak-256 hash of the batch ID rather than the text.
SAY THIS	"Hashes are one-way fingerprints; they make tampering detectable and identify transactions."
VIVA QUESTION	*Can you get the original data back from a hash?* No. A hash is one-way; you can only check whether some data produces the same hash.

2.2 Keys, addresses, signatures and wallets

Public key, private key and address

DEFINITION	A **private key** is a secret number only you hold. From it a **public key** is calculated, and from that an **address** (like `0x7099…79C8`), which is your public account identifier.
ANALOGY	The address is your house number, which anyone can know; the private key is the only key to your front door.
HOW IT WORKS	Ethereum uses elliptic-curve cryptography (secp256k1). The address is the last 20 bytes of the Keccak-256 hash of the public key. You can go from private key to address, but not backwards.
OUR PROJECT	The contract stores roles per **address**: `mapping(address => Role) _roles`. Whoever controls the private key of a Supplier address can act as the Supplier.

Public key, private key and address

SAY THIS	"A role belongs to an address. Whoever holds that address's private key can use the role, which is why private keys must be protected."
VIVA QUESTION	*What happens if a Supplier's private key leaks?* Anyone with it can sign as the Supplier. The Admin would need to revoke that address's role with `revokeRole`.

Digital signature

DEFINITION	Proof, made with a private key, that a specific account approved a specific message or transaction.
ANALOGY	A wax seal that only your ring can make, but anyone can recognise.
HOW IT WORKS	The wallet signs the transaction data with the private key. Anyone can check the signature against the data and recover the signer's address, without ever seeing the private key.
OUR PROJECT	In Solidity, the verified signer of a transaction becomes `msg.sender`. Every access check in our contract compares `msg.sender` with the role table.
SAY THIS	"The contract trusts `msg.sender` because it comes from a verified signature, not from anything typed into the UI."
VIVA QUESTION	*What is `msg.sender`?* The address that directly called the function. For a user transaction, it is the address whose key signed it.

Wallet (MetaMask)

DEFINITION	Software that stores your private keys and signs transactions when you approve them.
ANALOGY	A secure key-ring that asks "Do you really want to sign this?" every time.
HOW IT WORKS	MetaMask is a browser extension. It exposes an object, `window.ethereum`, that websites use to request accounts and ask for signatures. The private key never leaves MetaMask.
OUR PROJECT	The dashboard connects to MetaMask, reads the connected address, and asks MetaMask to sign every write. Chapter 9 covers it in depth.
SAY THIS	"MetaMask holds the keys and signs; my app never sees a private key."
VIVA QUESTION	*Does your app store private keys?* No. Only MetaMask holds them; the app receives addresses and signed results.

2.3 Smart contracts, the EVM and gas

Smart contract

DEFINITION	A program stored on the blockchain at its own address. Its functions run exactly as written whenever someone calls them.
ANALOGY	A vending machine: it follows fixed rules, and no shopkeeper can override them.

Smart contract

HOW IT WORKS	Written in **Solidity**, compiled to **bytecode**, and deployed by a transaction. It has storage (its own data) and functions. A failed rule check causes a **revert**: the whole transaction is undone.
OUR PROJECT	`BatteryPassport.sol`, deployed at `0x5fbdb2315678afecb367f032d93f642f64180aa3` on the local node.
SAY THIS	"The rules of the battery passport (who can record what, and in which order) live in the smart contract, not in the website."
VIVA QUESTION	*What does "revert" mean?* The transaction fails, all its changes are undone, and an error message is returned.

EVM and EVM-compatible networks

DEFINITION	The Ethereum Virtual Machine (EVM) is the "computer" that runs smart-contract bytecode. EVM-compatible networks can run the same contracts.
ANALOGY	A games console: a game made for it runs on any console of that type.
HOW IT WORKS	Every node runs the same bytecode and gets the same result, so they agree on the state.
OUR PROJECT	Hardhat's local network simulates an EVM chain. Our compile output reported *"evm target: osaka"*, the EVM version the bytecode targets.
SAY THIS	"Because it is standard EVM code, the same contract could in principle be deployed to another EVM network, for example a test network, without rewriting it."
VIVA QUESTION	*Is Hardhat a real blockchain?* It is a local development blockchain that behaves like Ethereum, run by one machine, for building and testing.

Gas and transaction fees

DEFINITION	**Gas** measures how much computation and storage a transaction uses; the **fee** is gas × gas price, paid in the network's currency (ETH).
ANALOGY	Fuel for a journey: longer journeys burn more fuel.
HOW IT WORKS	Writing to storage, especially long strings, costs a lot of gas. Reading with a call costs nothing because nothing is written.
OUR PROJECT	On Hardhat the ETH is **test ETH** with no real value; Hardhat gives its built-in accounts a large test balance. MetaMask still shows a fee estimate in test ETH.
SAY THIS	"Fees exist on a real network; on our local chain they are paid with worthless test ETH. Storing long text on-chain would be expensive in production."
VIVA QUESTION	*Why are reads free?* A read (`eth_call`) runs on one node without creating a transaction or block, so nothing is paid.

2.4 Talking to a blockchain: RPC and chain ID

RPC endpoint

DEFINITION	A URL where a node accepts requests. RPC means *Remote Procedure Call*: asking another computer to run a function for you. Ethereum nodes speak **JSON-RPC** (requests written in JSON, a text data format).
ANALOGY	The service counter of a post office.
HOW IT WORKS	Applications send requests such as `eth_call`, `eth_sendRawTransaction` or `eth_getTransactionReceipt`, usually over HTTP.
OUR PROJECT	The local node listens at `http://127.0.0.1:8545`. `127.0.0.1` means "this computer"; 8545 is the port. Both the dashboard and MetaMask talk to this URL.
SAY THIS	"The frontend talks to the blockchain through the node's JSON-RPC endpoint at 127.0.0.1, port 8545."
VIVA QUESTION	*Why can't a phone scanning your QR code read the passport?* 127.0.0.1 always means "the device itself", so the phone would look for a node on the phone.

Chain ID

DEFINITION	A number identifying which blockchain network you are on.
ANALOGY	A country's dialling code: the same phone number means different things in different countries.
HOW IT WORKS	Wallets put the chain ID into every signed transaction (EIP-155, an Ethereum Improvement Proposal), so a transaction signed for one network cannot be replayed on another.
OUR PROJECT	Hardhat's local chain uses **31337**. The dashboard checks that MetaMask is on 31337 and disables writes otherwise.
SAY THIS	"Chain ID 31337 identifies the local Hardhat network; the app refuses to send transactions on any other network."
VIVA QUESTION	*What happens if MetaMask is on another network?* The dashboard shows a warning, offers a "Switch / add Hardhat Local" button and keeps writes disabled.

2.5 Reading versus writing: the most important distinction

	Read: `eth_call`	Write: transaction
Purpose	Ask the current value of something.	Change the blockchain's state.
Signature	Not needed.	Required (MetaMask signs).
Creates a block?	No.	Yes, it is included in a block.
Cost	Free.	Gas fee (test ETH here).

	Read: `eth_call`	Write: transaction
Speed	Instant answer.	Wait for confirmation (instant on Hardhat, which automines).
Examples in our app	`getPublicPassport`, `getRole`, `getFullPassport`, `getAllBatchIds`.	`registerBattery`, `recordRefining`, …, `grantRole`.

This is why a customer can view a passport without MetaMask: viewing is a read, and reads need no signature.

2.6 Local blockchain versus public blockchain, and decentralisation

Decentralisation

DEFINITION No single party controls the system. Many independent nodes hold the data and agree on new blocks through a **consensus** mechanism.

ANALOGY Many independent notaries each keep a copy of the register and must agree before a new page is added.

HOW IT WORKS On a public network such as Ethereum, thousands of nodes validate each block. Rewriting history would require overpowering that network.

OUR PROJECT **Not decentralised.** One Hardhat node runs on one laptop, controlled by you, and it mines a block instantly for every transaction ("automine"). Restarting it erases everything. This is ideal for development and demonstration, but it is not a production network.

SAY THIS "My prototype demonstrates the smart-contract logic on a local single-node chain. Decentralisation and real tamper resistance would come from deploying it to a multi-party network."

VIVA QUESTION *Is your data really immutable?* On my local node I control the only copy, so it only demonstrates the principle. The contract itself offers no edit function; real immutability needs a network of independent nodes.

	Our local Hardhat network	A public network (e.g. Ethereum)
Who runs nodes	Only you	Thousands of independent operators
Money	Test ETH, no value	Real ETH, real fees
Persistence	Lost when the node stops	Permanent
Who can read	Anyone who can reach your node	Everyone in the world
Chain ID	31337	1 for Ethereum mainnet

A **permissioned** blockchain (e.g. Hyperledger Fabric) is a third option: only approved organisations run nodes and read data. Circulor's platform is publicly described as permissioned. Our project *imitates* permissions with role checks inside the contract, which is different (see Chapter 12).

2.7 Why blockchain cannot guarantee truth

This sentence is worth memorising: **"Blockchain guarantees the integrity of the record, not the truth of its content."** Our contract checks *who* submitted data, *when*, and that it follows the rules (non-empty text, recycled content between 0 and 100, carbon footprint greater than zero, correct stage order). It cannot check whether the mine really is where the supplier says, or whether the carbon figure is honest. This is often called the **oracle problem**: getting trustworthy real-world facts onto a blockchain.

What you can say

"Blockchain gives us accountability: we always know which account submitted each value, and it cannot be changed afterwards. But it does not prove the value was true. In a real system that needs audits, certificates or trusted measurement devices."

CHAPTER 3

The architecture of our project

Architecture means the parts of a system and how they talk to each other. Our system has five parts, and two separate routes to the blockchain: one for **writing** (needs MetaMask) and one for **reading** (does not).

3.1 The five parts

Part	What it is	Job in our project
User	A person using a web browser, optionally with the MetaMask extension.	Looks up passports; if authorised, records lifecycle stages.
React dashboard	A web page built with React, TypeScript and Vite (`frontend/`).	Shows passports, the QR code and role-based forms.
viem	A TypeScript library for talking to EVM blockchains.	Encodes function calls using the ABI, reads data, simulates calls, asks MetaMask to send transactions, and waits for receipts.
MetaMask	The wallet extension holding private keys.	Shows each transaction to the user, signs it and sends it to the node.
Hardhat node + `BatteryPassport.sol`	The local blockchain (chain ID 31337, `127.0.0.1:8545`) and our deployed contract.	Executes functions, enforces the rules, stores data, mines blocks.

3.2 The architecture diagram

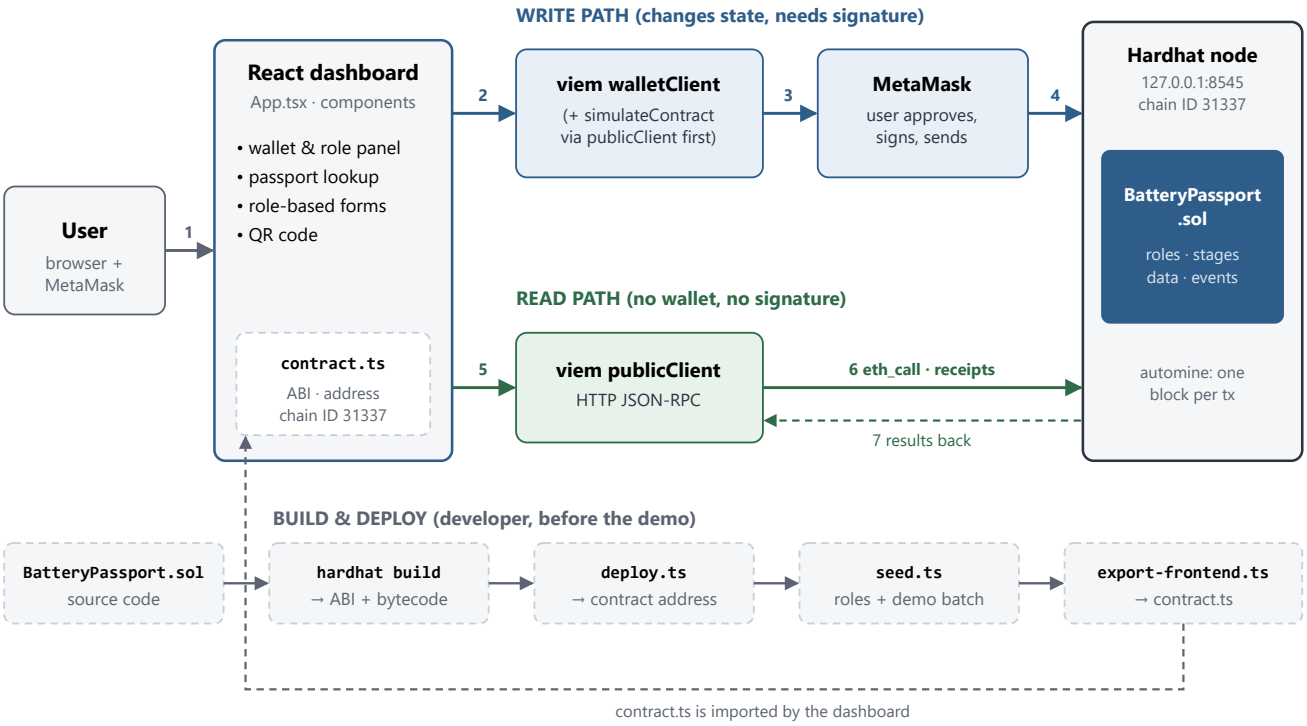


Figure 3.1 — System architecture. Blue = write path, green = read path, grey = developer build steps.

Every arrow explained

#	What travels along the arrow
1	The user opens `http://localhost:5173`, types a batch ID, clicks *Connect MetaMask*, or fills a form.
2	For a write, the dashboard calls viem. viem first runs `simulateContract` as the connected account (a free dry-run through the public client), then calls `writeContract` on the wallet client. viem turns "`registerBattery("DEMO-BATCH-0002", …)`" into encoded calldata using the ABI.
3	viem's wallet client sends an `eth_sendTransaction` request to MetaMask through `window.ethereum`. viem checks that MetaMask is on chain 31337 first.
4	MetaMask shows the request to the user. On approval, it fills in the nonce and gas, **signs** the transaction with the private key, and sends it to its configured network, Hardhat Local at `127.0.0.1:8545`. The node runs the contract function and mines a block.
5	For a read (public passport, role lookup) the dashboard calls viem's **public client**. No wallet is involved.
6	The public client sends JSON-RPC requests straight to the node over HTTP: `eth_call` for reads, `eth_getTransactionReceipt` while waiting for a write to confirm.
7	The node returns the result: passport data, the role number, or the receipt (block number, success/revert).

3.3 Who does what: viem versus MetaMask

Students often mix these two up. They have different jobs:

Job	viem	MetaMask
Knows the contract's functions (ABI)	Yes: encodes calls and decodes results	No
Reads blockchain data	Yes: public client over HTTP	Not used for reads in our app
Dry-runs a call to catch errors	Yes: `simulateContract`	No
Holds private keys	**No**	**Yes**
Asks the user for approval	No	Yes: the confirmation pop-up
Signs the transaction	No	Yes
Waits for the receipt	Yes: `waitForTransactionReceipt`	Shows its own activity list

A precise one-sentence version

"viem prepares and checks the call and later reads the result; MetaMask is the only component that holds the key and signs."

3.4 The write path step by step (a sequence diagram)

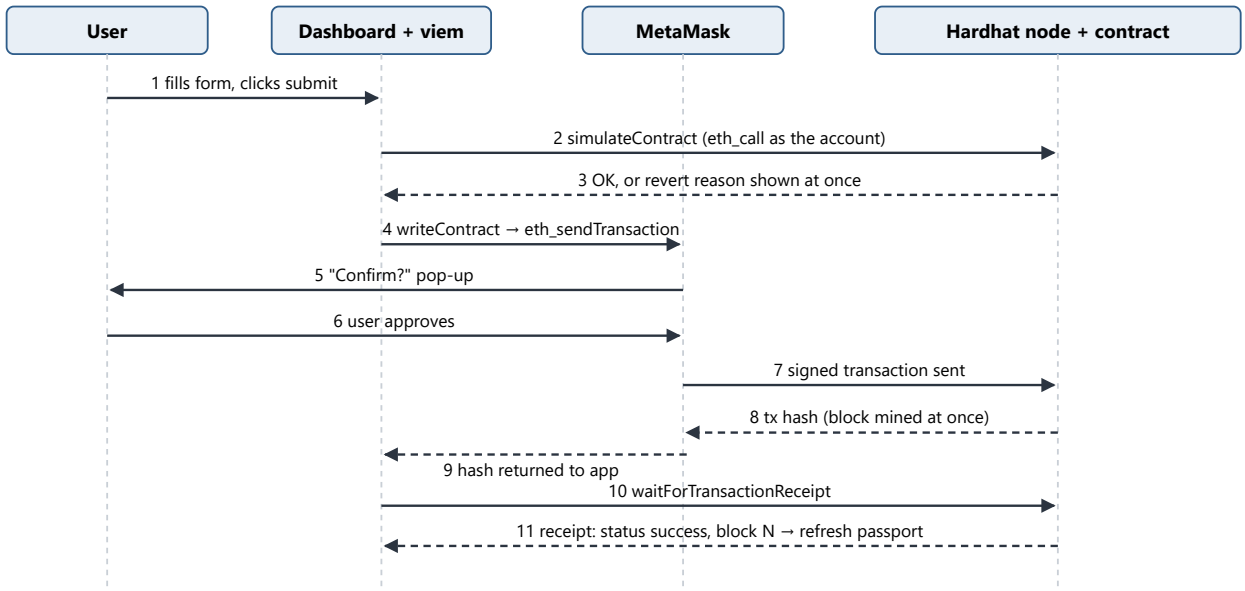


Figure 3.2 — What happens when a Supplier clicks "Register battery". Dashed arrows are responses.

3.5 The read path

When the page loads, or you click *View passport*, the dashboard calls:

```
publicClient.readContract({ ...contract, functionName: "getPublicPassport", args: [id] })
```

frontend/src/App.tsx, inside lookupBatch

That is a single `eth_call` to `http://127.0.0.1:8545`. There is no MetaMask pop-up, no fee, no signature and no block. This is what makes the customer view and the QR link work without a wallet.

3.6 Why this design? (decisions you can defend)

- **Two separate clients.** Reads should work even without MetaMask, so the app reads from the node directly. Writes must be signed by the user, so they go through MetaMask.
- **Simulate before sending.** A dry run shows the contract's own error (e.g. "Stage order violated…") before the user is asked to sign a transaction that would fail.
- **Wait for the receipt.** The UI only shows "Confirmed" after the node reports success, and then re-reads data from the chain. It never pretends a write succeeded.
- **The role comes from the contract.** The app reads `getRole(address)`; there is no role picker. Even if someone altered the website, the contract would still reject unauthorised writes.

Viva question

Why does the read path not use MetaMask? Reading needs no signature, and using a direct RPC connection lets customers view passports even if they do not have a wallet installed.

CHAPTER 4

Environment setup and project structure

The tools installed on your Windows laptop, what every folder and file in `C:\Users\divya\OneDrive\Documents\Project\VolvoBatteryPassport` does, and the commands we ran.

4.1 The tools

Node.js and npm

DEFINITION	**Node.js** runs JavaScript outside a browser. **npm** (Node Package Manager) downloads and manages libraries ("packages").
ANALOGY	Node.js is the engine; npm is the parts shop.
HOW IT WORKS	Each project lists its packages in `package.json`; `npm install` downloads them into `node_modules/` and records exact versions in `package-lock.json`. `npx` runs a tool from those packages.
OUR PROJECT	Node v24.22.0 and npm 11.19.1. Hardhat, the scripts and the frontend all run on Node.
SAY THIS	"Hardhat and the React toolchain are Node.js tools; npm manages their dependencies."
VIVA QUESTION	*What is `node_modules`?* The folder where npm installs dependencies; it is generated and not part of our own code.

TypeScript

DEFINITION	JavaScript with **types**: you declare what kind of value each variable holds, and mistakes are caught before running.
ANALOGY	Labelled containers in a kitchen: you notice before cooking that you grabbed salt instead of sugar.
HOW IT WORKS	The TypeScript compiler (`tsc`) checks types; tools such as Hardhat and Vite then run the code.
OUR PROJECT	All scripts (`scripts/*.ts`) and the frontend (`*.tsx`) are TypeScript. viem uses the ABI to type-check contract calls, so a wrong argument type is a compile error.
SAY THIS	"TypeScript plus viem means a wrong function name or argument type is caught before the app runs."
VIVA QUESTION	*What is a .tsx file?* A TypeScript file that can contain JSX, React's HTML-like syntax.

Hardhat

DEFINITION	A development environment for Ethereum smart contracts.
ANALOGY	A workshop with a test track: you build the car (compile), test it (tests), and drive it on a private track (local node).

Hardhat

HOW IT WORKS	Hardhat 3.20.0 compiles Solidity with solc 0.8.34, runs Solidity tests, runs a local blockchain (`npx hardhat node`) and runs scripts against a network (`npx hardhat run … --network localhost`).
OUR PROJECT	Configured by `hardhat.config.ts`; used for compile, tests, local node, deploy, seed and export.
SAY THIS	"Hardhat is my development framework: compiler, test runner and local blockchain in one tool."
VIVA QUESTION	*Why Hardhat 3 and not Hardhat 2?* It was the current version when the project was created. Hardhat 3 is ESM-only (modern JavaScript modules) and has a built-in Solidity test runner, which we used.

Vite and the development server

DEFINITION	Vite is a build tool for web apps. Its **development server** serves the app locally and reloads it when code changes.
ANALOGY	A rehearsal stage: you see changes immediately before the final performance (production build).
HOW IT WORKS	`npm run dev` serves the app at `http://localhost:5173`; `npm run build` type-checks and creates an optimised version in `frontend/dist/`.
OUR PROJECT	Vite 8.3.5 with the React plugin.
SAY THIS	"The dashboard is a React single-page app served locally by Vite."
VIVA QUESTION	*What does `npm run build` prove?* That the TypeScript compiles without errors and the app can be bundled. It does not prove the app behaves correctly in a browser.

4.2 The project folder

```
VolvoBatteryPassport/
├── contracts/BatteryPassport.sol       ← the smart contract (Chapter 5)
├── test/BatteryPassportTest.t.sol      ← 10 Solidity tests (Chapter 6)
├── scripts/
│   ├── deploy.ts                       ← deploys the contract (Chapter 7)
│   ├── seed.ts                         ← grants roles, creates DEMO-BATCH-0001
│   └── export-frontend.ts              ← writes ABI + address for the frontend
├── frontend/                           ← React + Vite app, own package.json (Chapter 8)
│   └── src/ App.tsx · chain.ts · contract.ts · index.css · main.tsx
│        └── components/ PassportView.tsx · ActionPanels.tsx
├── presentation/Volvo_Battery_Passport.pptx
├── artifacts/   cache/                 ← generated by compilation (not hand-written)
├── node_modules/                       ← installed packages (generated)
├── hardhat.config.ts  package.json  package-lock.json  tsconfig.json
└── README.md  .gitignore
```

The configuration files, exactly as they are

```
// hardhat.config.ts
import { defineConfig } from "hardhat/config";

export default defineConfig({
  solidity: {
    version: "0.8.34",
  },
});
```

This tells Hardhat one thing: compile with Solidity 0.8.34. No networks are listed because Hardhat 3 already provides a built-in simulated network and a `localhost` network pointing to `http://localhost:8545`. We confirmed this in Hardhat's own source code before writing the deploy script.

```
// package.json (root) — the important parts
"type": "module",
"devDependencies": {
  "@types/node": "^22.20.5",
  "forge-std": "github:foundry-rs/forge-std",
  "hardhat": "^3.20.0",
  "typescript": "~6.0.3"
}
```

`"type": "module"` means the project uses modern JavaScript modules (`import … from …`). `forge-std` is the Solidity testing library, installed directly from GitHub, which is why **Git** must be installed. The root has no ethers, viem or chai: we deliberately kept the Hardhat side minimal.

```
// tsconfig.json (root)
{
  "compilerOptions": { "lib": ["es2023"], "module": "node20", "target": "es2023",
    "skipLibCheck": true, "outDir": "dist", "types": ["node"], "verbatimModuleSyntax": true },
  "exclude": ["frontend", "node_modules"]
}
```

TypeScript settings for the scripts. We added the `exclude` line so the root type-check does not try to check the React frontend, which has its own settings (`frontend/tsconfig.app.json`).

The generated files

- **`artifacts/`**: the compiler output. `artifacts/contracts/BatteryPassport.sol/BatteryPassport.json` contains the **ABI** and **bytecode**; `artifacts/build-info/` contains the full compiler output (the seed script reads function selectors from it).
- **`frontend/src/contract.ts`**: generated by `scripts/export-frontend.ts`. It holds the contract address, chain ID 31337 and the ABI. Do not edit it by hand.

ABI (Application Binary Interface)

DEFINITION	A machine-readable list of a contract's functions, their inputs and outputs, and its events.
ANALOGY	The menu of a restaurant: it tells you what you can order and what you will get, but not how the kitchen works.

ABI (Application Binary Interface)

HOW IT WORKS	Tools like viem use the ABI to turn `registerBattery("X", "Y", "Z")` into the bytes the EVM understands, and to decode results back into readable values.
OUR PROJECT	The ABI has 17 entries: 12 functions, 1 constructor and 4 events. It is exported into `contract.ts` with `as const` so TypeScript knows every function's exact types.
SAY THIS	"The frontend uses the compiled ABI, so it always calls the contract's real functions with the right types."
VIVA QUESTION	*What is the difference between the ABI and bytecode?* The bytecode is the program the EVM runs; the ABI describes how to call it.

4.3 Commands we used

All commands run in PowerShell from the project root unless stated otherwise.

Command	Plain-language meaning	Expected output (observed)
`npm install`	Download the packages listed in `package.json`.	"added … packages"
`npm install --save-dev foundry-rs/forge-std`	Install the Solidity test library from GitHub (needs Git). Recorded in `package.json` as `github:foundry-rs/forge-std`.	Installed forge-std 1.17.0
`npx hardhat build`	Compile the contract and tests.	"Compiled 1 Solidity file with solc 0.8.34 (evm target: osaka)", or "No contracts to compile" if nothing changed
`npx hardhat test solidity`	Run the Solidity tests.	10 ✓ lines and "10 passing"
`npx tsc --noEmit`	Type-check the scripts without producing files.	No output = no errors
`npx hardhat node`	Start the local blockchain (keep this terminal open).	JSON-RPC server at 127.0.0.1:8545 and a list of test accounts
`npx hardhat run scripts/deploy.ts --network localhost`	Deploy the contract to the running node.	"BatteryPassport deployed to: 0x5fbd…0aa3"
`npx hardhat run scripts/seed.ts --network localhost`	Grant roles and create the demo batch.	"Final stage: Recycled (6)"
`npx hardhat run scripts/export-frontend.ts`	Copy the ABI and address into the frontend.	"Exported BatteryPassport ABI (17 entries)"
`npx --yes create-vite@latest frontend --template react-ts --no-interactive`	Create the React + TypeScript project (create-vite 9.3.1).	"Scaffolding project in … \frontend"

Command	Plain-language meaning	Expected output (observed)
`cd frontend` then `npm install` and `npm install viem qrcode.react`	Install the frontend's packages.	"found 0 vulnerabilities"
`npm run build` (in `frontend/`)	Type-check and build the dashboard.	"✓ built in …ms"
`npm run dev` (in `frontend/`)	Start the dashboard at http://localhost:5173.	"VITE v8.3.5 ready" and the local URL

The very first command that created the Hardhat project (Hardhat 3's "minimal" TypeScript template) was run before this record began; the resulting files are those shown above.

Installed versions

Root (blockchain side)	frontend/
hardhat 3.20.0 · typescript 6.0.3 · forge-std 1.17.0 · @types/node 22	react / react-dom 19.3.1 · vite 8.3.5 · @vitejs/plugin-react 6.1.2 · typescript 6.0.3 · viem 2.57.7 · qrcode.react 4.2.0 · @types/react(-dom) 19.3.0 · oxlint 1.89.0 (came with the template)

Viva question

Why does the frontend have its own package.json? To keep browser libraries (React, viem) separate from the Hardhat toolchain. Each side installs and builds independently.

CHAPTER 5

The smart contract, concept by concept

A walk through `contracts/BatteryPassport.sol` (490 lines, about half of them comments). Excerpts are copied from the file; the line numbers are the file's real line numbers.

5.1 Solidity in five minutes

Keyword	Meaning
`pragma solidity 0.8.34;`	"Compile this file only with Solidity version 0.8.34." (line 2)
`contract BatteryPassport { … }`	Declares the contract, like a class in other languages. (line 26)
`enum`	A fixed list of named options, stored as small numbers (0, 1, 2…).
`struct`	A group of named fields, like a form with several boxes.
`mapping(K => V)`	A lookup table from keys to values. Every possible key exists, with a default "zero" value until set.
`private`	Other contracts cannot read the variable directly. **It does not hide the data**; anyone with node access can read storage.
`external` / `view`	`external`: callable from outside. `view`: only reads, never changes state, so it can be called for free with `eth_call`.
`require(cond, "msg")`	If `cond` is false, stop and **revert** with the message. All changes in that call are undone.
`modifier`	A reusable check attached to functions; `_;` marks where the function body runs.
`msg.sender`	The address calling the function (the signer of the transaction).
`block.timestamp`	The time of the block the transaction is included in.
`event` / `emit`	A log entry written to the transaction receipt; cheap to store, readable off-chain.
`memory` / `storage`	`memory`: temporary data during the call. `storage`: the contract's permanent data.

5.2 The disclaimer at the top

Lines 4–25 are a NatSpec comment (Solidity's documentation-comment format) stating the two academic limitations: **access control is not confidentiality**, and **blockchain cannot verify data accuracy**. Putting them in the source shows the design was honest from the start; you can point to it in your viva.

5.3 Roles and stages: the two enums

```
    enum Role {
        None,         // 0 – no role; default for every address
        Admin,        // 1 – deploys contract, manages all roles
        Supplier,     // 2 – records mining, refining, end-of-life, recycling
        Manufacturer, // 3 – records manufacturing and distribution
        Regulator,    // 4 – read-only full audit access
        Customer      // 5 – read-only public passport access (role optional)
    }

    /// @dev Address → assigned Role
    mapping(address => Role) private _roles;
```

BatteryPassport.sol, lines 38–48

Every address starts as `None` (0), because a mapping returns the default value for keys it has never seen. That is why a brand-new MetaMask account shows "None" in the dashboard (Chapter 9). Each address holds **exactly one** role at a time.

```
    enum Stage {
        Unregistered, // 0 – not yet on chain
        Registered,   // 1 – mining / raw-material extraction recorded
        Refined,      // 2 – refining and processing recorded
        Manufactured, // 3 – cell manufacturing and vehicle assembly recorded
        Distributed,  // 4 – distribution and delivery recorded
        EndOfLife,    // 5 – end-of-life collection recorded
        Recycled      // 6 – recycling and material recovery recorded
    }
```

BatteryPassport.sol, lines 59–67

Design trick: `Unregistered` is value 0, the default. So "this batch ID has never been registered" is simply "its stage is still 0". No separate "exists" flag is needed.

5.4 The battery record: two structs

```
    struct Battery {
        // --- Mining / Registration ---
        string  batchId;         // unique batch identifier, e.g. "BATCH-2024-001"
        string  mineLocation;    // geographic origin, e.g. "Katanga, DRC"
        string  materialOrigin;  // comma-separated materials, e.g. "Cobalt, Lithium"
        uint256 registeredAt;    // block.timestamp of registration
        address registeredBy;    // supplier address that registered this batch
        // --- Refining ---
        string refiningDetails;    // description of the refining process
        uint8  recycledContentPct; // percentage of recycled material used (0–100)
        // --- Manufacturing ---
        string  manufacturingDetails; // assembly process description
        uint256 carbonFootprintKg;    // self-reported CO2-equivalent in kg
        string  vehicleVIN;           // Vehicle Identification Number
        bool    qualityPassed;        // result of quality inspection
        // --- Distribution ---
        string distributionDetails; // shipment and delivery description
        // --- Recycling ---
        string recoveredMaterials; // materials recovered at end of life
        // --- Lifecycle ---
        Stage currentStage;
    }
```

BatteryPassport.sol, lines 78–104 (blank lines removed)

`Battery` has 14 fields, the full record. `PublicPassport` (lines 112–124) has 11 fields: it leaves out `registeredAt`, `registeredBy` (the supplier's wallet address) and `refiningDetails`. Storage (lines 127 and 130):

```
    mapping(string => Battery) private _batteries;   // batch ID → record
    string[] private _batchIds;                       // list of every batch ID
```

Data types worth knowing

`uint8` is a whole number from 0 to 255, used for the percentage (the contract enforces ≤ 100). `uint256` is a very large whole number, used for the carbon footprint and timestamp. `bool` is true/false. `address` is a 20-byte account address. In the frontend, viem returns `uint8` values as JavaScript numbers and `uint256` values as `bigint`.

5.5 Events: the audit trail

```
    event RoleGranted(address indexed account, Role role, address indexed grantedBy);
    event RoleRevoked(address indexed account, address indexed revokedBy);
    event BatteryRegistered(string indexed batchId, string mineLocation, string materialOrigin,
                            address indexed registeredBy, uint256 timestamp);
    event StageAdvanced(string indexed batchId, Stage newStage, address indexed advancedBy,
                        uint256 timestamp);
```

BatteryPassport.sol, lines 137–157 (reformatted onto fewer lines)

Events are written into transaction receipts. They record *who did what, when*, and off-chain tools can search them. `indexed` fields can be filtered on. **Detail an examiner may ask about:** because `batchId` is an indexed *string*, the log stores the Keccak-256 hash of the batch ID, not the readable text.

NOT TESTED Our tests do not assert that events are emitted, and the dashboard does not display event history. The events exist in the code but are not demonstrated.

5.6 The constructor: who becomes Admin?

```
    constructor() {
        _roles[msg.sender] = Role.Admin;
        emit RoleGranted(msg.sender, Role.Admin, msg.sender);
    }
```

BatteryPassport.sol, lines 166–169

A constructor runs **once**, when the contract is deployed. Whoever deploys it becomes Admin. Our `deploy.ts` deploys from the node's first account (Hardhat account #0, `0xf39F…2266`), so that account is Admin.

5.7 Modifiers: the access and order checks

```
    modifier onlySupplier() {
        require(
            _roles[msg.sender] == Role.Supplier,
            "Access denied: Supplier role required"
        );
        _;
    }
```

BatteryPassport.sol, lines 183–189

All six modifiers follow this pattern. Run the check; if it passes, `_;` continues into the function.

Modifier (line)	Checks	Revert message
`onlyAdmin` (175)	caller's role is Admin	Access denied: Admin role required
`onlySupplier` (183)	caller's role is Supplier	Access denied: Supplier role required
`onlyManufacturer` (191)	caller's role is Manufacturer	Access denied: Manufacturer role required
`onlyAdminOrRegulator` (199)	caller is Admin or Regulator	Access denied: Admin or Regulator role required
`batteryExists` (212)	the batch's stage is not Unregistered	Battery not found: batch ID does not exist
`atStage` (224)	the batch is at exactly the expected stage	Stage order violated: prerequisite stage not yet complete

```
    modifier atStage(string memory batchId, Stage expected) {
        require(
            _batteries[batchId].currentStage == expected,
            "Stage order violated: prerequisite stage not yet complete"
        );
        _;
    }
```

BatteryPassport.sol, lines 224–230

Access control with modifiers

DEFINITION	Restricting which addresses may call which functions.
ANALOGY	A security guard checking your staff badge at a door.
HOW IT WORKS	Modifiers run before the function body, in the order written. For `recordRefining`: role check, then existence check, then stage check, then the input checks inside the body. The first failure reverts the whole call.
OUR PROJECT	Every write function has a role modifier. Every lifecycle function after registration also has `batteryExists` and `atStage`.
SAY THIS	"The contract checks `msg.sender` against its own role table; the UI cannot grant or fake permissions."
VIVA QUESTION	*In what order are the checks applied, and does it matter?* Left to right as written: role first. That is why test 6 (refining an unregistered batch as a valid Supplier) gets "Battery not found" rather than "Stage order violated": `batteryExists` runs before `atStage`.

5.8 Function by function

Each card answers: what it does, who can call it, inputs, what changes on-chain, what happens if the call is invalid, and where it is demonstrated.

grantRole (line 241)

```
function grantRole(address account, Role role) external onlyAdmin {
    require(account != address(0), "Invalid address: zero address not allowed");
    require(role != Role.None,     "Use revokeRole to remove a role");
    require(uint8(role) <= 5,      "Invalid role: value must be between 1 and 5");
    _roles[account] = role;
    emit RoleGranted(account, role, msg.sender);
}
```

Does	Assigns a role to an address, replacing any role it had.
Who	Admin only.
Inputs	`account` (address), `role` (1–5).
Changes	`_roles[account]`; emits `RoleGranted`.
If invalid	Non-admin: "Access denied: Admin role required". Zero address, or `None`: specific messages. Note: the `≤ 5` check is a belt-and-braces check; Solidity's input decoding already rejects enum values outside 0–5 before the function body runs.
Shown in	Tests 2 and 3 VERIFIED; `seed.ts` granted roles to accounts #1–#4 VERIFIED; dashboard Admin form NOT YET VERIFIED.

revokeRole (line 253) and getRole (line 265)

revokeRole	Admin only. Sets an address back to `None` and emits `RoleRevoked`. Reverts with "Address has no role to revoke" if the address has no role, or on the zero address. NOT TESTED, NOT YET USED IN THE UI

getRole	Anyone; `view`. Returns the role number (0–5) of any address. The dashboard calls it to decide which forms to show. VERIFIED (dashboard showed Supplier for the imported Supplier account; tests 1 and 2).

registerBattery (line 281): the most important function

```
    function registerBattery(
        string memory batchId,
        string memory mineLocation,
        string memory materialOrigin
    ) external onlySupplier {
        require(bytes(batchId).length > 0,        "Validation: batchId cannot be empty");
        require(bytes(mineLocation).length > 0,   "Validation: mineLocation cannot be empty");
        require(bytes(materialOrigin).length > 0, "Validation: materialOrigin cannot be empty");
        require(
            _batteries[batchId].currentStage == Stage.Unregistered,
            "Duplicate: a battery with this batch ID already exists"
        );

        Battery storage b = _batteries[batchId];
        b.batchId        = batchId;
        b.mineLocation   = mineLocation;
        b.materialOrigin = materialOrigin;
        b.registeredAt   = block.timestamp;
        b.registeredBy   = msg.sender;
        b.currentStage   = Stage.Registered;

        _batchIds.push(batchId);

        emit BatteryRegistered(batchId, mineLocation, materialOrigin, msg.sender, block.timestamp);
        emit StageAdvanced(batchId, Stage.Registered, msg.sender, block.timestamp);
    }
```

BatteryPassport.sol, lines 281–306

Reading it line by line: only a Supplier may call it (`onlySupplier`). The three `require` lines reject empty text: `bytes(x).length > 0` is how Solidity checks that a string is not empty. The fourth `require` rejects a duplicate ID, using the stage-0 trick. `Battery storage b` is a *pointer* to the record in permanent storage, so assigning to `b.…` writes on-chain. The time and the caller's address are recorded automatically; the caller cannot fake them. The ID is added to the list, and two events are emitted.

If invalid	Wrong role → "Access denied: Supplier role required". Empty field → "Validation: … cannot be empty". Existing ID → "Duplicate: a battery with this batch ID already exists".
Shown in	Tests 4, 5, 8 VERIFIED. Seed: DEMO-BATCH-0001 VERIFIED. Dashboard: DEMO-BATCH-0002 registered by the Supplier, confirmed in block 12 VERIFIED.

recordRefining (line 316)

```
    function recordRefining(
        string memory batchId,
        string memory refiningDetails,
        uint8         recycledContentPct
    ) external onlySupplier batteryExists(batchId) atStage(batchId, Stage.Registered) {
        require(bytes(refiningDetails).length > 0, "Validation: refiningDetails cannot be empty");
        require(recycledContentPct <= 100,         "Validation: recycledContentPct must be between 0
and 100");

        Battery storage b = _batteries[batchId];
        b.refiningDetails    = refiningDetails;
        b.recycledContentPct = recycledContentPct;
        b.currentStage       = Stage.Refined;

        emit StageAdvanced(batchId, Stage.Refined, msg.sender, block.timestamp);
    }
```

BatteryPassport.sol, lines 316–330

The pattern for every later stage is identical: role modifier, existence, correct current stage, input checks, write fields, move the stage forward by one, emit `StageAdvanced`.

Shown in: test 6 (refining before registration is rejected) and tests 9–10 (successful refining) VERIFIED; dashboard: DEMO-BATCH-0002 advanced to Refined VERIFIED.

The remaining lifecycle functions

Function (line)	Who	Inputs → stored	Requires stage	Moves to
`recordManufacturing` (342)	Manufacturer	`manufacturingDetails` (non-empty), `carbonFootprintKg` (uint256, must be > 0), `vehicleVIN` (non-empty), `qualityPassed` (bool)	Refined	Manufactured
`recordDistribution` (370)	Manufacturer	`distributionDetails` (non-empty)	Manufactured	Distributed
`recordEndOfLife` (389)	Supplier	batch ID only; stage change	Distributed	EndOfLife
`recordRecycling` (403)	Supplier	`recoveredMaterials` (non-empty)	EndOfLife	Recycled

Shown in: test 7 (manufacturing before refining rejected) and test 9 (full lifecycle) VERIFIED; seed ran all four for DEMO-BATCH-0001 VERIFIED; dashboard forms for these stages PENDING, not yet exercised with MetaMask (Chapter 11).

Why does the Supplier do end-of-life and recycling?

The project uses five roles only (agreed for the two-day MVP). Collection and recycling are material-handling activities, so they were grouped with the Supplier, reflecting the circular idea that recovered materials flow back to suppliers. A production system would probably add a separate Recycler role. This is a simplification you should mention if asked.

5.9 The lifecycle as a state machine

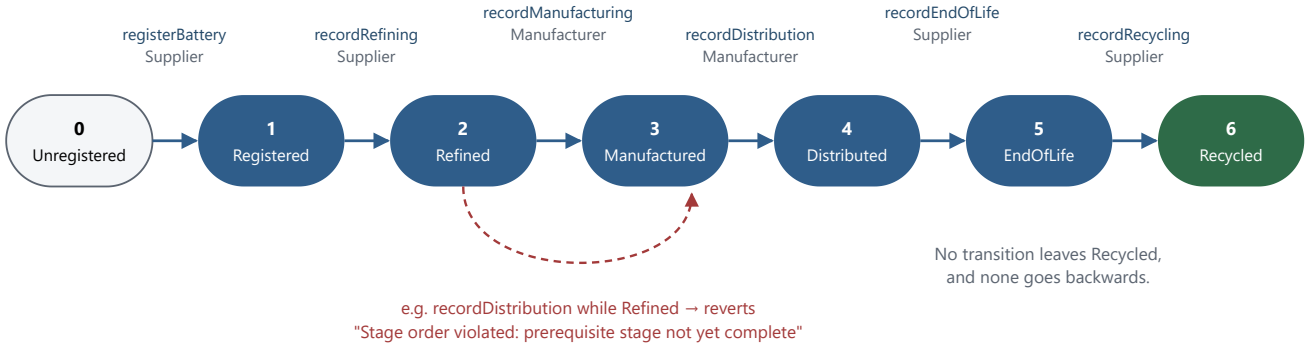


Figure 5.1 — The six-stage lifecycle. Every arrow is exactly one function, callable by exactly one role, from exactly one stage.

State machine

DEFINITION	A system that is always in one of a fixed set of states and can move only along allowed transitions.
ANALOGY	A train line: you can only go to the next station, never skip one or go back.
HOW IT WORKS	`currentStage` holds the state; `atStage(expected)` blocks any transition from the wrong state.
OUR PROJECT	Each stage function can succeed only once per batch, so recorded data cannot be overwritten by calling it again.
SAY THIS	"The lifecycle is enforced as a state machine in the contract, so stages cannot be skipped, repeated or reversed."
VIVA QUESTION	*Can the Supplier call recordRefining twice to change the recycled percentage?* No. After the first call the stage is Refined, so a second call fails the `atStage(Registered)` check.

5.10 Reading data: public versus restricted

Function (line)	Who	Returns	Shown in
`getPublicPassport` (433)	Anyone, no role, no wallet	11 public fields, including `currentStage`. Reverts "Battery not found" for unknown IDs.	Tests 4, 9; dashboard lookup and QR VERIFIED
`getFullPassport` (467)	Admin, Regulator	All 14 fields, adding supplier address, registration time and refining details.	Test 10 VERIFIED; dashboard Audit panel NOT YET VERIFIED
`getAllBatchIds` (482)	Admin, Regulator	Array of every registered batch ID, in registration order.	Dashboard Audit panel NOT YET VERIFIED; no contract test

During development, a viem script read the deployed contract directly over RPC. It confirmed that the Regulator account could read the full passport and batch list, and that the Customer account's request was rejected with "Access denied: Admin or Regulator role required". That checked the contract and the read code path, not the dashboard screen.

Restricted ≠ private

`getFullPassport` is a `view` function, so it is called with `eth_call`, and in an `eth_call` the caller can claim any `from` address, including the Regulator's. On top of that, anyone with node access can read raw contract storage. So this restriction is a **function-level access rule, not confidentiality**. Write functions are different: there `msg.sender` is proven by a signature.

5.11 Input validation summary

Rule	Where	Message
Text fields must not be empty	every write function with text inputs	Validation: <field> cannot be empty
Batch ID must be new	registerBattery	Duplicate: a battery with this batch ID already exists
Recycled content ≤ 100	recordRefining	Validation: recycledContentPct must be between 0 and 100
Carbon footprint > 0	recordManufacturing	Validation: carbonFootprintKg must be greater than zero
No zero address; no granting None; revoke only if a role exists	grantRole / revokeRole	(specific messages, see 5.8)

The contract does not check formats, such as whether a VIN has 17 characters, whether a location is real, or whether text is reasonable. This is basic validation suitable for a prototype.

CHAPTER 6

Testing the contract

`test/BatteryPassportTest.t.sol` contains 10 tests written in Solidity using the forge-std library. All 10 pass with `npx hardhat test solidity`.

6.1 Why Solidity tests, and how they run

Hardhat 3 can run tests written in Solidity directly in its EVM. We chose this because the project has no JavaScript testing libraries installed (no ethers, chai or mocha). Hardhat's own documentation recommends Solidity tests for unit-testing a single contract, and it needed only one extra package, `forge-std`. Hardhat 3.20.0 has built-in support for resolving `import "forge-std/Test.sol"`; we confirmed this in Hardhat's resolver code before writing the test.

Cheatcodes: vm.prank and vm.expectRevert

DEFINITION	Special test-only commands that change how the next call behaves.
ANALOGY	`vm.prank` is an actor wearing another person's badge for one scene; `vm.expectRevert` is a director saying "in the next scene, the guard must stop you, with these exact words".
HOW IT WORKS	`vm.prank(addr)` makes the **next** call arrive with `msg.sender = addr`. `vm.expectRevert(bytes("msg"))` makes the test fail unless the next call reverts with exactly that message.
OUR PROJECT	Used to act as Supplier, Manufacturer, Regulator, Customer or an outsider, and to check exact error messages.
SAY THIS	"I used vm.prank to simulate different callers and vm.expectRevert to check the exact revert reason."
VIVA QUESTION	*Do the tests use real private keys?* No. `vm.prank` sets `msg.sender` directly inside the test EVM; no signatures are involved.

6.2 The test setup

```
contract BatteryPassportTest is Test {
    BatteryPassport internal passport;

    address internal admin;
    address internal supplier = makeAddr("supplier");
    address internal manufacturer = makeAddr("manufacturer");
    address internal regulator = makeAddr("regulator");
    address internal customer = makeAddr("customer");
    address internal outsider = makeAddr("outsider");

    string internal constant BATCH = "BATCH-2026-001";

    function setUp() public {
        passport = new BatteryPassport();
        admin = address(this);
    }
```

test/BatteryPassportTest.t.sol, lines 9–24

`setUp()` runs before **every** test, so each test starts with a fresh contract. The test contract itself deploys the passport, so the test contract (`address(this)`) is the Admin. `makeAddr("supplier")` creates a fixed, labelled test address. These are simulated test addresses, not the Hardhat accounts used in MetaMask. Two helpers (lines 26–36) grant all roles and register a batch as the supplier.

6.3 The ten tests

#	Test (line)	What it does and expects	Proves
1	`test_DeploymentAssignsAdminToDeployer` (39)	Reads `getRole(admin)`; expects Admin.	Constructor
2	`test_AdminCanGrantSupplier` (44)	Admin grants Supplier; expects the role to read back as Supplier.	Role granting
3	`test_NonAdminCannotGrantRoles` (50)	An outsider calls `grantRole`; expects "Access denied: Admin role required" and that the role stays None.	Access control
4	`test_SupplierCanRegisterBattery` (59)	Supplier registers; public passport shows the batch ID, mine location, material origin, stage Registered.	Registration
5	`test_UnauthorizedCannotRegisterBattery` (71)	An outsider **and** the Manufacturer try to register; both get "Access denied: Supplier role required".	Access control
6	`test_RefiningBeforeRegistrationRejected` (85)	Supplier refines an unregistered batch; expects "Battery not found: batch ID does not exist".	Existence check
7	`test_ManufacturingBeforeRefiningRejected` (94)	Batch registered but not refined; Manufacturer tries manufacturing; expects "Stage order violated...".	Stage order
8	`test_DuplicateBatchIdRejected` (104)	Registers the same ID twice; second gets "Duplicate: ...".	Uniqueness
9	`test_PublicPassportAfterFullLifecycle` (114)	Runs all six stages with the correct roles; an outsider reads the public passport; checks all 11 fields and stage Recycled.	Full lifecycle
10	`test_RegulatorCanReadFullPassportCustomerCannot` (151)	Regulator reads the full passport (checks `registeredBy`, refining details, stage Refined); Customer gets "Access denied: Admin or Regulator role required".	Restricted read

Example: test 3, a failure test

```
    function test_NonAdminCannotGrantRoles() public {
        vm.expectRevert(bytes("Access denied: Admin role required"));
        vm.prank(outsider);
        passport.grantRole(outsider, BatteryPassport.Role.Supplier);

        assertEq(uint8(passport.getRole(outsider)), uint8(BatteryPassport.Role.None));
    }
```

test/BatteryPassportTest.t.sol, lines 50–56

Read it as: "expect the next call to revert with this message; make the next call come from the outsider; try to grant a role; afterwards the outsider must still have no role." The outsider tried to give *itself* the Supplier role, which is exactly the attack the check exists to stop.

Example: test 7, a stage-order test

```
    function test_ManufacturingBeforeRefiningRejected() public {
        _grantAllRoles();
        _register(BATCH);

        vm.expectRevert(bytes("Stage order violated: prerequisite stage not yet complete"));
        vm.prank(manufacturer);
        passport.recordManufacturing(BATCH, "Cell and pack assembly", 45, "YV1EX90TEST00001", true);
    }
```

test/BatteryPassportTest.t.sol, lines 94–101

6.4 The actual output

```
Compiled 1 Solidity file with solc 0.8.34 (evm target: osaka)

Running Solidity tests

  test/BatteryPassportTest.t.sol:BatteryPassportTest
    ✓ test_UnauthorizedCannotRegisterBattery()
    ✓ test_SupplierCanRegisterBattery()
    ✓ test_RegulatorCanReadFullPassportCustomerCannot()
    ✓ test_RefiningBeforeRegistrationRejected()
    ✓ test_PublicPassportAfterFullLifecycle()
    ✓ test_NonAdminCannotGrantRoles()
    ✓ test_ManufacturingBeforeRefiningRejected()
    ✓ test_DuplicateBatchIdRejected()
    ✓ test_DeploymentAssignsAdminToDeployer()
    ✓ test_AdminCanGrantSupplier()

  10 passing
```

Tests may print in a different order each run; order does not matter because each test starts from a fresh contract.

6.5 What passing tests prove, and what they do not

Established by the 10 tests

- The deployer becomes Admin; only the Admin can grant roles.
- Only a Supplier can register; outsiders and Manufacturers are rejected.
- Unregistered batches cannot be refined.
- Manufacturing before refining is rejected.
- Duplicate IDs are rejected.
- A correct full lifecycle produces the expected public passport.
- Regulator can and Customer cannot read the full passport.
- Revert messages match exactly.

Not covered by the tests

- Events being emitted with correct values.
- Empty-field, percentage > 100 and zero-carbon validation.
- `revokeRole`, zero-address checks, `getAllBatchIds`.
- Wrong-role or wrong-stage calls for distribution, end of life and recycling individually.
- Calling a stage twice.
- The frontend, MetaMask, or the deployed local node (tests use Hardhat's in-memory test EVM).
- Gas costs and security against all attacks.

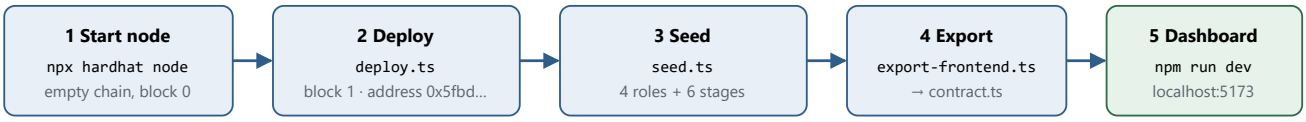
What you can say

"Ten unit tests cover the core rules: role assignment, access control on registration and role management, existence and stage-order checks, duplicates, the full lifecycle, and restricted reads. They do not cover every validation message or the user interface; those were checked manually, and some UI checks are still pending."

CHAPTER 7

Deployment, seeding and frontend export

How the contract gets from a source file onto a running local blockchain, how the demo data and roles are created, and how the dashboard learns where the contract lives.



Steps 3 and 4 can run in either order; both need step 2's address.

Figure 7.1 — From an empty local chain to a working dashboard.

7.1 Starting the local blockchain

```
npx hardhat node
```

This starts a single-node Ethereum-compatible blockchain in memory, with chain ID 31337 and a JSON-RPC server at `http://127.0.0.1:8545`. It prints a list of pre-funded **test accounts** with their private keys; these keys are the same on every Hardhat installation and are therefore **public**. The node **automines**: each transaction is put into its own new block straight away. Keep this terminal open for the whole demo.

7.2 Deploying the contract: `scripts/deploy.ts`

Deploying means sending a special transaction that has no recipient and carries the contract's bytecode. The node runs the constructor and stores the code at a new **contract address**.

```
const { provider, networkName, networkConfig } = await network.create();

// Safety guard: this educational prototype is only deployed to local networks.
if (networkName !== "localhost" && networkConfig.type !== "edr-simulated") {
  throw new Error(`Refusing to deploy to "${networkName}". …`);
}

const artifact = await artifacts.readArtifact("BatteryPassport");
const accounts = (await provider.request({ method: "eth_accounts" })) as Address[];
const deployer = accounts[0];

const txHash = (await provider.request({
  method: "eth_sendTransaction",
  params: [{ from: deployer, data: artifact.bytecode }],
})) as Address;
```

scripts/deploy.ts (excerpt; a comment and an empty-accounts check removed)

Step by step: connect to the network chosen with `--network`; refuse anything that is not local; load the compiled bytecode from `artifacts/`; take the node's first account (#0) as the deployer; send the deployment transaction. The script then waits for the receipt, checks `status` is `0x1` (success), checks that code really exists at the new address (`eth_getCode`), and prints the result.

Who signs the deployment? Not MetaMask.

The Hardhat node itself holds the keys of its built-in test accounts, so it can sign `eth_sendTransaction` requests from those accounts. That is why the scripts work without MetaMask. In the dashboard, MetaMask does the signing instead.

Why does the root project not use ethers or viem?

To keep the Hardhat side minimal, as agreed in the plan, no extra libraries were installed there. The scripts therefore talk to the node with plain JSON-RPC requests through Hardhat's built-in provider. This is more manual, but fully supported by Hardhat 3.

Output on the local node VERIFIED (you ran this):

```
Network:  localhost
Deployer: 0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 (becomes Admin in the constructor)
Block:    1

BatteryPassport deployed to: 0x5fbdb2315678afecb367f032d93f642f64180aa3
```

Contract address

DEFINITION	The address where the deployed contract's code and storage live; every call must be sent to it.
ANALOGY	The street address of a shop. The shop's goods (data) are only at that address.
HOW IT WORKS	It is calculated from the deployer's address and the deployer's transaction count (nonce). That is why a fresh node always gives the same address when account #0 deploys first: `0x5fbd…0aa3`.
OUR PROJECT	Stored in `frontend/src/contract.ts` and used as the default in `seed.ts`.
SAY THIS	"The contract address is deterministic on a fresh node, but the data behind it is lost whenever the node restarts."
VIVA QUESTION	*What happens if the frontend has the wrong address?* Calls go to an address with no contract code. The dashboard reports "No contract responded at …" and suggests redeploying and re-exporting.

7.3 Seeding roles and demo data: `scripts/seed.ts`

```
npx hardhat run scripts/seed.ts --network localhost
```

The seed script prepares the demonstration. VERIFIED

1. **Safety checks:** it runs only on `localhost`, checks that contract code exists at the address, and checks that account #0 really is Admin.
2. **Grants roles** from account #0, skipping any already granted:

Hardhat account	Address	Role granted
#0	`0xf39F…2266`	Admin (from deployment)
#1	`0x7099…79C8`	Supplier
#2	`0x3C44…93BC`	Manufacturer
#3	`0x90F7…b906`	Regulator
#4	`0x15d3…6A65`	Customer

3. **Runs `DEMO-BATCH-0001` through all six stages**, each from the correct account, with fictional data: mine "Demo Mine Site A (fictional location)", materials "Cobalt, Lithium, Nickel (demo data)", recycled content 16 %, carbon footprint 5,600 kg CO_2e, VIN `DEMOVIN0000000001`, quality passed, shipment to a fictional dealership in Gothenburg, recovered lithium, cobalt and nickel.
4. After each transaction it waits for a successful receipt and checks the stage moved forward by exactly one.
5. It prints `Batch ID: DEMO-BATCH-0001` and `Final stage: Recycled (6)`.

Safe to rerun. It first reads the batch's current stage. If the batch exists, it skips registration and continues from the next missing stage, so it never triggers the duplicate error. If a stage fails to advance, it stops and lists the stages it did not run.

How it calls the contract without a library: every function call must be encoded as a 4-byte **function selector** followed by the ABI-encoded arguments. The script reads the selectors from Hardhat's compiler output (`artifacts/build-info/…output.json`, field `evm.methodIdentifiers`) instead of hard-coding them, then encodes strings, numbers, booleans and addresses itself. This is the most technical file in the project; you only need to understand the idea.

> **Verified state after seeding**
>
> A direct read of the contract after seeding returned `DEMO-BATCH-0001` at stage 6 (Recycled) with the expected fields, and roles 1, 2, 3, 4, 5 for accounts #0–#4. The chain was at block 11: block 1 for deployment, 4 role grants, 6 lifecycle transactions.

7.4 Exporting to the frontend: `scripts/export-frontend.ts`

```
npx hardhat run scripts/export-frontend.ts
```

This script does not connect to any network. It reads the compiled ABI through Hardhat's artifact API, validates the address format (`0x` + 40 hex characters), and writes `frontend/src/contract.ts`:

```
// AUTO-GENERATED by scripts/export-frontend.ts. Do not edit by hand.
// The address below is only valid for the current local Hardhat node. …
export const BATTERY_PASSPORT_ADDRESS = "0x5fbdb2315678afecb367f032d93f642f64180aa3" as const;
export const CHAIN_ID = 31337 as const;
export const BATTERY_PASSPORT_ABI = [ … 17 entries … ] as const;
```

Output: `Exported BatteryPassport ABI (17 entries)` VERIFIED. An invalid address such as `0x123` was also tested: the script stops with "Invalid contract address" and writes nothing.

The default address can be overridden with an environment variable, for both `seed.ts` and `export-frontend.ts`:

```
$env:BATTERY_PASSPORT_ADDRESS="0xNewAddress"
npx hardhat run scripts/export-frontend.ts
npx hardhat run scripts/seed.ts --network localhost
```

7.5 Why the frontend must use the right address and ABI

- **Wrong address:** calls reach an empty address and return no data. viem raises a "zero data" error, which the dashboard translates into "No contract responded at …".
- **Out-of-date ABI** (if the contract were changed and recompiled but not re-exported): the frontend would encode calls for functions that no longer match. Always re-export after recompiling.

7.6 Restarting the node: what is lost and what to repeat

Stopping `npx hardhat node` erases everything

The local chain lives only in memory. Stopping it deletes the contract, all roles, `DEMO-BATCH-0001`, and `DEMO-BATCH-0002`. **Do not restart the node before you finish the remaining demo steps in Chapter 11 and take your screenshots**, or you will need to register and refine `DEMO-BATCH-0002` again.

After a restart, repeat in order:

1. `npx hardhat run scripts/deploy.ts --network localhost`. The address is normally the same `0x5fbd…0aa3`, because account #0 deploys first.
2. `npx hardhat run scripts/export-frontend.ts`, needed only if the address changed, but harmless to run.
3. `npx hardhat run scripts/seed.ts --network localhost`
4. In MetaMask, clear activity data for each imported account (Settings → Advanced → *Clear activity tab data*). MetaMask remembers transaction counts (nonces) from the old chain and would otherwise report "nonce too high".

CHAPTER 8

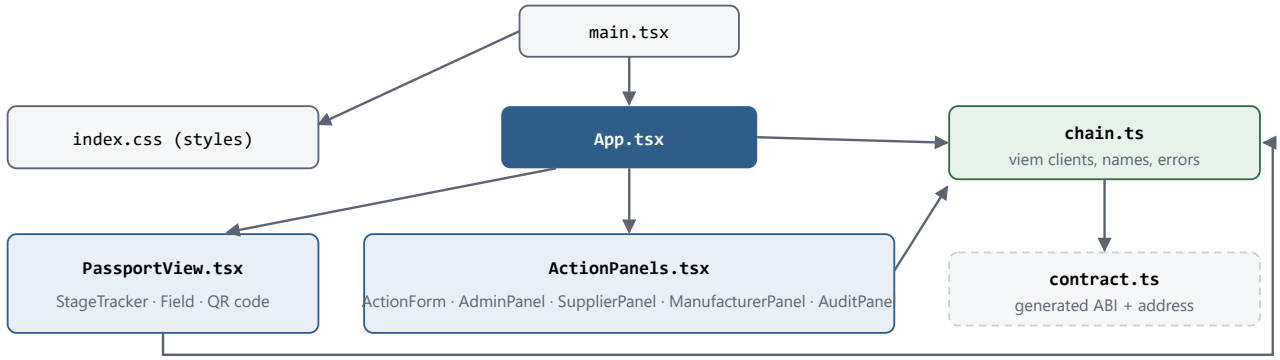
Frontend walkthrough

How the React dashboard is built, file by file, and what happens on screen at each step. Only the React ideas needed for this project are explained.

8.1 Just enough React

Idea	Meaning	Example in our code
Component	A function that returns a piece of the page.	`PassportView` draws one passport.
Props	Inputs passed into a component by its parent.	`<PassportView passport={passport} />`
State (`useState`)	Values a component remembers; changing them redraws the page.	`account`, `role`, `passport`, `tx`
Effect (`useEffect`)	Code that runs after drawing, e.g. to fetch data or subscribe to events.	Re-read the role whenever the account changes.
`async` / `await`	Waiting for slow operations (network calls) without freezing the page.	`await publicClient.readContract(…)`

8.2 How the files fit together



Arrows mean "uses / imports". App.tsx renders the two component files; App, PassportView and ActionPanels all use chain.ts, which imports contract.ts.

Figure 8.1 — Frontend file structure.

8.3 `chain.ts`: the blockchain toolkit

```
export const publicClient = createPublicClient({ chain: localChain, transport: http(RPC_URL) });

export const contract = { address: BATTERY_PASSPORT_ADDRESS, abi: BATTERY_PASSPORT_ABI } as const;

export function makeWalletClient(account: Address, provider: EIP1193Provider) {
  return createWalletClient({ account, chain: localChain, transport: custom(provider) });
}
```

frontend/src/chain.ts, lines 30–36

- `publicClient`: the read-only client. It talks directly to `http://127.0.0.1:8545` using viem's built-in `hardhat` chain definition (ID 31337).
- `makeWalletClient`: creates the write client for a connected account. Its transport is `custom(provider)`, where `provider` is MetaMask's `window.ethereum`, so requests go to MetaMask.
- `ROLE_NAMES`, `STAGE_NAMES`: copies of the contract's enums in the same order (0 = None…, 0 = Unregistered…), used to show names instead of numbers.
- `NEXT_STEP`: the text "Next step: Record manufacturing (Manufacturer)" shown under the tracker.
- `describeError` turns errors into readable messages. A contract revert shows its reason, e.g. "Access denied: …". A MetaMask rejection shows "Request rejected in MetaMask." No contract at the address shows "No contract responded at …". A node that cannot be reached shows "Cannot reach the local Hardhat node … Is `npx hardhat node` running?"
- **Types** `PublicPassport` and `FullPassport` are derived from the ABI, so TypeScript knows every field.

8.4 `App.tsx`: the single dashboard

`App` holds the page's state: `account`, `walletChainId`, `role`, `batchInput`, `passport`, `lookup` (loading or error) and `tx` (the transaction banner). It draws four cards: **Wallet & on-chain role**, **Public battery passport**, **Actions for your role** and **About this data**.

Wallet and role

- On load it asks MetaMask (if present) for already-connected accounts and the chain ID, and subscribes to `accountsChanged` and `chainChanged` (line 59), so switching accounts in MetaMask updates the page automatically.
- `connect()` (line 92) calls `eth_requestAccounts`, which opens MetaMask's "connect to this site" prompt.
- Whenever the account changes, an effect calls `getRole(account)` through the public client and stores the answer. **That number from the contract is the only source of the displayed role.**
- If MetaMask is not on 31337, a warning appears with a *Switch / add Hardhat Local* button (`switchToLocal`, line 104). It uses `wallet_switchEthereumChain`, or `wallet_addEthereumChain` if MetaMask doesn't know the network yet.

Passport lookup

- `lookupBatch` (line 139) trims the ID, calls `getPublicPassport` and stores the result. It also rewrites the browser URL to `?passport=<id>` (line 149), so the address bar always matches the QR link.
- On page load it reads `?passport=` from the URL, or uses `DEMO-BATCH-0001` as the default example, and looks it up automatically. The values shown always come from the chain, never from hard-coded text.
- An unknown ID shows: *No battery with batch ID "…" is registered on this contract.*

Writes: the `submit` function

```
  const submit: Submit = async (label, batchId, write) => {
    if (!account || !wallet) {
      setTx({ label, phase: "error", message: "Connect MetaMask first." });
      return false;
    }
    if (!onLocalChain) {
      setTx({ label, phase: "error", message: `Switch MetaMask to the local Hardhat network (chain
ID ${CHAIN_ID}).` });
      return false;
    }
    setTx({ label, phase: "signing" });
    try {
      const hash = await write({ account, wallet });
      setTx({ label, phase: "mining", hash });
      const receipt = await publicClient.waitForTransactionReceipt({ hash });
      if (receipt.status !== "success") throw new Error(`Transaction ${hash} reverted.`);
      setTx({ label, phase: "success", hash, block: receipt.blockNumber });
      setRoleRefresh((n) => n + 1);
      if (batchId) await lookupBatch(batchId);
      return true;
    } catch (error) {
      setTx({ label, phase: "error", message: describeError(error) });
      return false;
    }
  };
```

frontend/src/App.tsx, lines 181–204

Every form uses this one function. The banner moves through: "*Checking with the contract, then confirm in MetaMask…*" → "*Transaction sent. Waiting for the receipt…*" → "*Confirmed in block N.*" (or the error message). After success it re-reads the role and the passport **from the blockchain**.

8.5 `ActionPanels.tsx`: role-based forms

A reusable `ActionForm` component draws a titled form from a list of fields and runs simple checks before anything is sent. The panels:

Panel	Shown to	Forms
`AdminPanel`	Admin	Grant role (address + role list), Revoke role. Checks the address format first.
`SupplierPanel`	Supplier	1 · Register battery · 2 · Record refining (0–100 %) · 5 · Record end of life · 6 · Record recycling
`ManufacturerPanel`	Manufacturer	3 · Record manufacturing (carbon footprint must be a whole number > 0; quality checkbox) · 4 · Record distribution
`AuditPanel`	Admin, Regulator	Load full passport · List all batch IDs (click one to open its public passport)

Customers and accounts with no role see an explanation instead of forms. The batch-ID field of each stage form (except Register) is pre-filled with the batch currently displayed.

```
          await submit("Register battery", id, async (ctx) => {
            const { request } = await publicClient.simulateContract({
              ...contract,
              functionName: "registerBattery",
              args: [id, text(v, "mineLocation"), text(v, "materialOrigin")],
              account: ctx.account,
            });
            return ctx.wallet.writeContract(request);
          });
```

frontend/src/components/ActionPanels.tsx, lines 212–220

The two steps of every write are visible here: `simulateContract` (a free dry run as your account, which surfaces revert reasons) and `writeContract` (hand the transaction to MetaMask).

> **Two layers of validation, and why both exist**
>
> Some checks happen in the browser first, e.g. "Recycled content must be a whole number from 0 to 100." These give quick feedback and never reach the blockchain. The contract repeats the important checks itself. **Only the contract's checks are security**: anyone could bypass a website and call the contract directly.

8.6 `PassportView.tsx`: the passport card and QR code

- `StageTracker`: six numbered steps (Registered … Recycled); completed steps show ✓ and the current step is highlighted.
- **Fields** in three groups: *Material origin*, *Manufacturing history*, *Distribution & end of life*. A field from a stage not yet reached shows "*Not yet recorded*" instead of an empty or zero value.
- **Carbon footprint** is labelled "(self-reported)" on screen, reflecting the data-truth limitation.
- **QR code** drawn by `QRCodeSVG` from `qrcode.react`, encoding:

```
export function passportUrl(batchId: string): string {
  return `${window.location.origin}${window.location.pathname}?
passport=${encodeURIComponent(batchId)}`;
}
```

frontend/src/components/PassportView.tsx, lines 38–40

When the dashboard is served by `npm run dev`, this produces `http://localhost:5173/?passport=DEMO-BATCH-0001`. `encodeURIComponent` makes IDs with spaces or symbols safe in a URL. The QR code is generated entirely in the browser; no external service is used.

8.7 `contract.ts`, `index.css` and `main.tsx`

- `contract.ts`: generated address, chain ID and ABI (Chapter 7).
- `index.css`: all styling. It defines a light palette (charcoal text, muted blue accents, green for success, red for errors), the card layout, the stage tracker, the transaction banner colours and a narrow-screen layout below 720 px.
- `main.tsx`: the Vite template's entry point, which mounts `App` into the page.

8.8 The user experience, start to finish

1. **Open** `http://localhost:5173`. The demo passport loads straight away, with no wallet needed.
2. **Read a passport:** type a batch ID and click *View passport* (or *Demo batch*).
3. **Connect MetaMask:** click *Connect MetaMask* and approve the connection.
4. **See account and role:** shortened address, network (green "Hardhat Local (31337)" or an amber warning), role badge read from the contract.
5. **Fill in a form** for your role.
6. **Sign:** the app simulates first; if the contract would reject the call, the error appears and MetaMask never opens. Otherwise MetaMask shows the transaction; approve it.
7. **Wait for the receipt:** the banner shows the transaction hash, then "Confirmed in block N."
8. **See the refreshed passport** with the new stage and fields, read again from the chain.
9. **QR code:** open or scan the link on the same computer to see the same public passport.

CHAPTER 9

MetaMask and local test accounts

Most demo problems are wallet problems: wrong network, wrong account, or a stale nonce. This chapter explains how MetaMask fits in and how to diagnose each problem.

Golden rule

Never paste a private key into a chat, a document, a screenshot or a presentation, not even a test key. Keys go only into MetaMask's own *Import account* screen. This guide deliberately contains no private keys.

9.1 How the dashboard finds MetaMask

MetaMask injects an object called `window.ethereum` into every web page. The dashboard reads it once: `const provider = window.ethereum;`. If it is missing, the wallet card shows *"MetaMask was not detected. You can still look up public passports below…"*.

Why was MetaMask "not detected" at first? The exact cause in your case was not recorded. These are the usual causes, in order of likelihood:

1. The extension was not installed or enabled in the browser (or browser profile) used to open `localhost:5173`.
2. The page was opened before the extension was installed or enabled. Refresh the page.
3. The browser's extension settings blocked MetaMask's access to the site. In Chrome or Edge, check the extension's "site access" setting.

Public passports still work without MetaMask, because reads go straight to the node (Chapter 3).

9.2 Adding and switching to Hardhat Local

Field in MetaMask	Value	Why
Network name	Hardhat Local	Any label you like
RPC URL	`http://127.0.0.1:8545`	Where `npx hardhat node` listens
Chain ID	`31337`	Hardhat's local chain ID; must match exactly
Currency symbol	ETH	Test ETH only

MetaMask may warn that chain ID 31337 is also used by other networks or that the symbol does not match a known network. For a local test network this warning is expected. The dashboard's *Switch / add Hardhat Local* button can also add the network for you. VERIFIED MetaMask was connected to Hardhat Local.

9.3 Why a brand-new MetaMask account could not do anything

A newly created MetaMask account has a fresh, random address. Two things are wrong with it for our demo:

1. **No role.** The contract has never heard of this address, so `_roles[address]` returns the default, `None` (0). The dashboard shows role **None** and the message "*This account has no role on the contract. The Admin must grant one before it can record data.*"
2. **No test ETH.** On Hardhat Local only the built-in test accounts start with ETH. A new account cannot pay gas, so even with a role it could not send a transaction unless someone sent it test ETH first.

The simplest fix, and the one we used, is to **import the Hardhat test accounts** that `seed.ts` already gave roles to.

9.4 Importing a known Hardhat test account

1. Look at the terminal running `npx hardhat node`; it lists accounts #0, #1, … with addresses and private keys.
2. In MetaMask: account menu → *Add account or hardware wallet* → *Import account*. The menu wording can differ slightly between MetaMask versions.
3. Paste the private key of the account you need, for example #1 for Supplier. Never paste it anywhere else.
4. Select that account in MetaMask. The dashboard re-reads its role automatically.

VERIFIED The Supplier account (#1, `0x7099…79C8`) was imported and the dashboard showed role **Supplier**.

Account	Address (shortened)	Role after seeding	Use it for
#0	`0xf39F…2266`	Admin	Grant/revoke roles; audit view
#1	`0x7099…79C8`	Supplier	Register, refine, end of life, recycle
#2	`0x3C44…93BC`	Manufacturer	Manufacture, distribute
#3	`0x90F7…b906`	Regulator	Full passport, list of batches
#4	`0x15d3…6A65`	Customer	Public passport only

9.5 How the contract decides your role

The dashboard sends `getRole(yourAddress)`; the contract looks up `_roles[yourAddress]` and returns a number, 0–5. When you submit a form, the contract does **the same lookup again** on `msg.sender` inside the modifier. The website never decides permissions. If someone edited the website to show Supplier forms to a Customer, the contract would still revert.

Why separate accounts per role?

Each address holds exactly one role, and each lifecycle step belongs to one role. Using different accounts shows **separation of duties**: the Supplier cannot record manufacturing, the Manufacturer cannot register batches, and only the Admin manages roles. In the real world each would be a different organisation with its own keys.

9.6 Why public test keys must never hold real value

Hardhat's default accounts are generated from a mnemonic phrase published in Hardhat's own documentation, so **everyone in the world knows their private keys**. Automated bots watch real networks and immediately take any funds sent to these addresses. Use them only on your local chain. If you ever use

MetaMask with real assets, keep those accounts separate, or use a separate browser profile for development.

9.7 Diagnosing common wallet problems

Symptom	Likely cause	Fix
"MetaMask was not detected"	Extension missing or disabled in this browser	Install or enable it, then refresh
Amber "Chain ID ..." warning; buttons disabled	MetaMask on another network	Click *Switch / add Hardhat Local*
Role shows **None**	New or unrelated account selected	Select an imported Hardhat account, or have the Admin grant a role
Wrong role shown	Wrong account selected in MetaMask	Switch account; the role refreshes automatically
"Cannot reach the local Hardhat node..."	`npx hardhat node` not running	Start it (then redeploy and seed if it was restarted)
"No contract responded at 0x..."	Node restarted; contract gone	Deploy, export, seed again
"Nonce too high", or a transaction stuck	MetaMask remembers an old chain's transaction count	MetaMask → Settings → Advanced → *Clear activity tab data*
"Request rejected in MetaMask."	You clicked Reject	Submit again and approve
Insufficient funds	Account has no test ETH	Use an imported Hardhat account

CHAPTER 10

Demonstration: our actual progress

The exact working sequence you carried out, with what you should see at each step, what it proves and what commonly goes wrong. Status labels reflect the milestones recorded during the project.

10.1 The existing example: `DEMO-BATCH-0001`

VERIFIED Created by `seed.ts` using the node's built-in accounts (not MetaMask). It went through all six stages and is at **Recycled (6)**. Its public passport shows fictional data: Demo Mine Site A, cobalt/lithium/nickel, 16 % recycled content, 5,600 kg CO_2e, VIN `DEMOVIN0000000001`, quality passed, shipped to a fictional Gothenburg dealership, and recovered lithium, cobalt and nickel. It proves the **whole lifecycle** works on the deployed contract and gives the dashboard something to show on first load.

`DEMO-BATCH-0002` is the **live** example. It was created through the dashboard with MetaMask, which demonstrates the user-facing write path.

10.2 The sequence, step by step

#	Step	What you should see	What it proves	Status
1	Compile	`npx hardhat build`: "Compiled 1 Solidity file with solc 0.8.34" or "No contracts to compile".	The contract is valid Solidity.	DONE
2	Run tests	`npx hardhat test solidity`: "10 passing".	Core rules behave as designed.	DONE
3	Start local blockchain	`npx hardhat node`: server at 127.0.0.1:8545 and an account list.	A chain is available.	DONE
4	Deploy	"BatteryPassport deployed to: 0x5fbd...0aa3", block 1.	Contract live; account #0 is Admin.	DONE
5	Seed roles + demo data	Role grants, six "OK" lines, "Final stage: Recycled (6)".	Roles assigned; full lifecycle works on the live contract.	DONE
6	Export ABI and address	"Exported BatteryPassport ABI (17 entries)".	Frontend points at the right contract.	DONE
7	Start frontend	`npm run dev` in `frontend/`; DEMO-BATCH-0001 passport and QR on `localhost:5173`.	Read path and QR generation work.	DONE
8	Connect MetaMask, switch to Hardhat Local	Network shows "Hardhat Local (31337)" in green.	Wallet integration and chain check.	DONE
9	Connect Supplier account	Role badge: **Supplier**; Supplier forms appear.	Role read from the contract, not chosen in the UI.	DONE
10	Register DEMO-BATCH-0002	MetaMask confirmation, then "Confirmed in block 12."	A user-signed write passed the contract's checks.	DONE

#	Step	What you should see	What it proves	Status
11	Verify its public passport	Tracker at step 1, Registered; origin fields filled; later fields "Not yet recorded".	The write is readable by anyone.	DONE
12	Record refining	Form 2 with your refining details and recycled-content %.	Second lifecycle transition.	DONE
13	Confirm the transaction	MetaMask approval, then "Confirmed in block N".	Receipt-based confirmation.	DONE
14	Verify status is Refined	Tracker at step 2, Refined; "Next step: Record manufacturing (Manufacturer)".	State machine advanced exactly one stage.	DONE

The exact refining text and percentage you entered for DEMO-BATCH-0002 were not recorded in this guide; check them on the passport before presenting. The refining transaction's block number was also not recorded. On an untouched node it would be block 13, but that is an inference, not a verified fact.

10.3 Common errors at these steps

- **Step 2:** an import error for `forge-std` means the package is missing. Run `npm install`, which needs Git.
- **Step 4:** "Refusing to deploy…" means `--network localhost` was forgotten or misspelled.
- **Step 5:** "No contract at 0x…" means the node was restarted after deploying, or the address changed.
- **Step 7:** "Cannot reach the local Hardhat node" means the node terminal was closed.
- **Steps 9–10:** role None (wrong account), amber chain warning (wrong network), or "Duplicate: a battery with this batch ID already exists" (the ID was already used).
- **Step 12:** "Recycled content must be a whole number from 0 to 100." is the browser check; the contract has its own matching check.

CHAPTER 11

Remaining work and completion checklist

What still needs to be done before the project is "finished": the remaining stages of DEMO-BATCH-0002 through the dashboard, the role views that have not yet been exercised, negative tests in the UI, and evidence for the presentation. None of this has been done yet.

Before you start

Keep the same `npx hardhat node` running. If it was stopped, DEMO-BATCH-0002 no longer exists; redeploy, re-export and re-seed (Chapter 7), then register and refine DEMO-BATCH-0002 again as in Chapter 10. Then start here.

11.1 Finish the lifecycle of DEMO-BATCH-0002 PENDING

Use clearly fictional values. The forms exist in the code (`ManufacturerPanel`, `SupplierPanel`) but have **not yet been exercised with MetaMask**.

#	MetaMask account	Dashboard form	Example inputs (fictional)	Verify afterwards
A	#2 Manufacturer	3 · Record manufacturing	Details "Demo cell and pack assembly"; carbon 4,800; VIN `DEMOVIN0000000002`; quality ticked	Stage **Manufactured**; carbon, VIN and quality shown; next step: distribution
B	#2 Manufacturer	4 · Record distribution	"Demo shipment to a fictional dealership"	Stage **Distributed**; distribution details shown
C	#1 Supplier	5 · Record end of life	Batch ID only	Stage **End of Life**; "Collected for recycling"
D	#1 Supplier	6 · Record recycling	"Demo recovery: lithium, cobalt (illustrative)"	Stage **Recycled**; tracker all ✓; "Lifecycle complete"

For each step, record the block number from the "Confirmed in block N" banner. It is useful evidence for your results slide.

11.2 Exercise the other role views PENDING

Account	Action	Expected result
#3 Regulator	*List all batch IDs*	DEMO-BATCH-0001 and DEMO-BATCH-0002 (if the node was not restarted)
#3 Regulator	*Load full passport* for DEMO-BATCH-0002	*Registered by* = the Supplier's address (#1, `0x7099…79C8`); registration time; refining details

Account	Action	Expected result
#4 Customer	Connect	Role Customer; no forms, no audit panel; public passport still visible
#0 Admin (optional)	Grant a role to an extra Hardhat account (e.g. #5), switch to it, then revoke it	Role appears, then returns to None. This would be the first test of `revokeRole` anywhere in the project.

11.3 Show that invalid actions are rejected PENDING

Try this	Expected (from the code)	Proves
As #2, at Refined, submit **4 · Record distribution** before manufacturing	Banner shows "Stage order violated: prerequisite stage not yet complete"; MetaMask never opens	Stage order (contract) + simulation
As #1, register `DEMO-BATCH-0001` again	"Duplicate: a battery with this batch ID already exists"	Uniqueness
As #1, refining with recycled content 150	"Recycled content must be a whole number from 0 to 100." (browser check; never reaches the chain)	Client-side validation
As #2, carbon footprint 0	"Carbon footprint must be a whole number greater than zero." (browser check)	Client-side validation
Look up `NOPE-123`	"No battery with batch ID "NOPE-123" is registered on this contract."	Missing-batch handling
Switch MetaMask to another network	Amber warning; write buttons disabled	Chain check

> **About wrong-role attempts in the UI**
>
> The dashboard shows only your own role's forms, so you cannot click "Register battery" as a Manufacturer. Wrong-role rejection is therefore evidenced by contract tests 3, 5 and 10, which check the exact messages, not by the UI. Say this honestly if asked.

11.4 Final consistency checks PENDING

☐ **Roles:** accounts #0–#4 show Admin, Supplier, Manufacturer, Regulator, Customer respectively.

☐ **Lifecycle order:** DEMO-BATCH-0002's tracker advanced one step per transaction, ending at Recycled.

☐ **Passport contents:** each field matches what you entered; fields from stages not yet reached showed "Not yet recorded" earlier.

☐ **Unauthorised calls rejected:** the stage-order and duplicate attempts in 11.3 failed with the expected messages.

☐ **QR lookup:** opening `http://localhost:5173/?passport=DEMO-BATCH-0002` in a new tab shows the same passport.

☐ **Frontend–contract consistency:** the address in `frontend/src/contract.ts` equals the address printed by `deploy.ts`.

☐ **Tests still pass:** rerun `npx hardhat test solidity` → 10 passing.

11.5 Evidence to collect for the presentation PENDING

- ☐ Screenshot: public passport of DEMO-BATCH-0001 with tracker and QR code (for slide 11's placeholder).
- ☐ Screenshot: DEMO-BATCH-0002 at Recycled after you finish 11.1.
- ☐ Screenshot: wallet card showing account, "Hardhat Local (31337)" and a role badge.
- ☐ Screenshot: MetaMask confirmation pop-up for one transaction. Make sure no private key is visible.
- ☐ Screenshot: a rejected action (stage-order error banner).
- ☐ Screenshot: Regulator full-passport view.
- ☐ Screenshot or copy of the terminal: "10 passing".
- ☐ A note of the block numbers for each DEMO-BATCH-0002 transaction.

When everything in Chapter 11 is done, update the "Results" slide of your presentation: it currently says DEMO-BATCH-0002 reached Refined.

CHAPTER 12

Security, limitations and future improvements

Examiners respect honest limitations. This chapter keeps three things apart: what is implemented, what its weaknesses are, and what a production system would add.

12.1 Limitations of the current prototype

Limitation	Explanation
Fictional data	All batches, mines, VINs and figures are invented for demonstration. Nothing describes a real battery.
Truth of submitted data	The contract records who submitted what, but cannot know whether it is true. A dishonest supplier can permanently record false values (the "oracle problem").
Public visibility	Everything stored is readable by anyone who can query the node, including "private" variables and the full passport.
Local, single node	One machine, one operator, automine. No decentralisation, and all data is lost when the node stops.
No corrections	A mistaken entry cannot be fixed; the stage has moved on and no amendment function exists.
Simplified roles	Five roles; Supplier also handles end of life and recycling. One role per address.
Basic validation	Non-empty text and simple ranges only; no format checks (e.g. VIN length).
Same-computer QR codes	QR links point to `localhost` and the dashboard reads `127.0.0.1:8545`, so a phone cannot open them.
Untested areas	Events, several validation messages, `revokeRole` and `getAllBatchIds` have no automated tests; some UI flows are still pending (Chapter 11).

12.2 Access control is not confidentiality

Access control versus confidentiality

DEFINITION

Access control decides who may *perform an action*. **Confidentiality** decides who may *see information*.

ANALOGY

A glass office with a locked door: only staff can go in and move things, but everyone outside can see through the walls.

HOW IT WORKS

Our modifiers stop unauthorised *transactions*. But contract storage is part of the blockchain state that every node holds, and anyone with access to the node can read it with standard RPC calls. And because view functions are called with `eth_call`, where the caller can claim any `from` address, the check on `getFullPassport` can be bypassed.

OUR PROJECT

Writes are genuinely protected, because the signature proves `msg.sender`. The "restricted" full passport is only a convenience separation, not privacy. The dashboard says so in the *About this data* card and next to the full-passport view.

Access control versus confidentiality

SAY THIS	"Role checks protect who can change data. They do not hide data; for confidentiality you would keep sensitive data off-chain or use a permissioned network."
VIVA QUESTION	*The variables are marked private. Isn't the data private?* No. `private` only stops other contracts from reading the variable directly; the raw storage is still visible to anyone reading the chain.

12.3 Security considerations

- **Private-key safety.** Whoever holds a role's key *is* that role. Hardhat's test keys are public and must never hold real value. In production, organisations would use hardware security modules or multi-signature wallets.
- **Single-admin risk.** One Admin address controls all roles. If its key is stolen, the attacker can grant itself any role; if it is lost, or the Admin revokes its own role (the contract allows this), nobody can manage roles again.
- **No upgrade path.** A deployed contract cannot be changed. A bug would require deploying a new contract and migrating data.
- **Simulation is a convenience, not a guarantee.** `simulateContract` improves the user experience; the real checks still happen when the transaction executes.
- **Client-side checks are not security.** Browser validation can be bypassed; the contract's `require` statements are what matter.

12.4 Scalability and on-chain storage trade-offs

- **Strings cost gas.** Each text field is stored in full on-chain. On a public network, storing descriptions and locations as text would be expensive.
- **Growing list.** `getAllBatchIds` returns the whole array; with very many batches the call becomes slow or exceeds node limits. Real systems page through results or use an off-chain indexer that reads events.
- **Hashed event keys.** Because `batchId` is an indexed string in events, logs store its hash; an indexer would need the original IDs from elsewhere to display them.

12.5 What a production deployment would need FUTURE WORK: NONE IMPLEMENTED

Area	Possible improvement
Network	A permissioned network shared by the participating companies (e.g. Hyperledger Fabric or a private EVM network), or a public test network for a multi-device demo.
Confidentiality	Keep sensitive documents off-chain; store only their hashes on-chain so they can be verified without being revealed.
Data quality	Supplier audits, signed certificates from accredited bodies, measurement devices (IoT), and anomaly detection.
Identity	Link each address to a verified organisation (e.g. digital certificates); key management per company.

Area	Possible improvement
Governance	Multiple admins or multi-signature approval; role expiry; an established access-control library (e.g. OpenZeppelin AccessControl).
Corrections	An amendment mechanism that adds a correction record, keeping the original visible.
Passport content	Battery health, repair history, compliance certificates, an event-history view in the UI.
Hosting	Dashboard served over HTTPS at a public URL so QR codes work from any phone.
Testing	Tests for events, every validation rule, `revokeRole`, batch listing; end-to-end UI tests; a security review.

CHAPTER 13

Viva and presentation preparation

Scripts to practise aloud, the questions you are most likely to face, the follow-ups that test real understanding, statements to avoid, and a glossary.

13.1 The 2–3 minute explanation

"EV batteries are made from materials that pass through many companies: miners, refiners, battery makers, car makers, and finally recyclers. Each keeps separate records, so it is hard to trace where a battery came from or to check claims like recycled content and carbon footprint. EU regulation now requires a digital battery passport for EV batteries from 2027.

Inspired by the Volvo Cars and Circulor traceability work, I built an educational prototype of a blockchain battery passport. A single Solidity smart contract stores battery batches and enforces five roles (Admin, Supplier, Manufacturer, Regulator and Customer) and six lifecycle stages from Registered to Recycled. Only the right role can record each stage, and only in the correct order. Every update is a transaction signed in MetaMask, so each record is tied to an account and cannot be changed afterwards.

A React dashboard lets anyone look up a public passport without a wallet, and generates a QR code linking to it. Authorised participants connect MetaMask, and the dashboard reads their role from the contract and shows only their forms. Ten automated tests check the access rules and the lifecycle order. Everything runs on a local Hardhat blockchain with fictional data.

The key lesson is that blockchain gives accountability and tamper-evidence, but it cannot prove submitted data is true, and access control is not the same as privacy."

13.2 The 5-minute technical walkthrough

1. **(30 s) Problem and goal.** Traceability across many parties; prototype goal.
2. **(60 s) Architecture.** Show Figure 3.1: React → viem → MetaMask → Hardhat node → contract for writes; React → viem public client → node for reads.
3. **(90 s) Contract.** Role and Stage enums; `mapping(address => Role)`; modifiers `onlySupplier`, `batteryExists`, `atStage`; walk through `registerBattery`; events.
4. **(45 s) Tests.** Ten Solidity tests with `vm.prank` and `vm.expectRevert`; examples: non-admin cannot grant roles, manufacturing before refining fails.
5. **(60 s) Live demo.** Public passport without wallet → QR link → connect Supplier → register or advance a batch → MetaMask confirm → "Confirmed in block N" → passport refreshes.
6. **(15 s) Limitations.** Local chain, data truth, public visibility.

13.3 Explaining the architecture in one breath

"The user works in a React dashboard. To change data, the dashboard uses viem to prepare and simulate the call, and MetaMask signs and sends it to the local Hardhat node, where BatteryPassport.sol checks the sender's role and the batch's stage before storing anything. To read data, the dashboard uses viem's public

client to call the node directly, so no wallet is needed. The ABI and address come from a script that exports them after deployment."

13.4 Viva questions and answers

1. What problem does your project solve?
It shows how a battery batch's lifecycle, from mining to recycling, can be recorded by different participants in one shared, tamper-evident record, with a public passport anyone can view.

2. Why use blockchain instead of a central database?
Participants who do not fully trust each other can share one record that no single party can quietly edit. Each entry is signed, and the rules are enforced by code everyone can inspect. A database owner could alter records. The trade-off: blockchain does not make data true or private.

3. What is a smart contract?
A program deployed on a blockchain whose functions run exactly as written. Ours, BatteryPassport.sol, stores batches and enforces roles and stage order.

4. Which Solidity version and why pin it exactly?
0.8.34, pinned with `pragma solidity 0.8.34;` and in hardhat.config.ts, so the contract always compiles with the same compiler that was tested. Solidity 0.8 also checks arithmetic overflow automatically.

5. How are roles stored and checked?
`mapping(address => Role) _roles`. Modifiers such as `onlySupplier` compare `_roles[msg.sender]` with the required role and revert otherwise.

6. Who is the Admin and how did they get the role?
The deploying account (Hardhat account #0). The constructor sets `_roles[msg.sender] = Role.Admin`.

7. What is msg.sender and why can you trust it?
The address calling the function. For a transaction it comes from the verified digital signature, so it cannot be faked without the private key.

8. How do you enforce the lifecycle order?
Each stage function has `atStage(expected)`, which requires `currentStage` to equal the previous stage. Calls out of order revert with "Stage order violated: prerequisite stage not yet complete".

9. How do you prevent duplicate batch IDs?
`registerBattery` requires the batch's stage to still be Unregistered (0, the default). If it already exists, it reverts with "Duplicate: a battery with this batch ID already exists".

10. What happens when a transaction reverts?
All of its changes are undone and an error message is returned; nothing is stored. The dashboard shows the reason, often already during simulation, before MetaMask opens.

11. What does Hardhat do in your project?
It compiles the contract, runs the 10 Solidity tests, runs the local blockchain (chain ID 31337), and runs the deploy, seed and export scripts.

12. What is MetaMask's role?
It holds the private keys, shows each transaction to the user for approval, signs it and sends it to the Hardhat node. The app never sees a private key.

13. What is viem's role?

It encodes and decodes contract calls using the ABI, reads data over RPC, simulates writes, hands transactions to MetaMask, and waits for receipts.

14. What is an RPC endpoint? Which one do you use?

A URL where a node accepts JSON-RPC requests. Ours is `http://127.0.0.1:8545`, the local Hardhat node.

15. Why chain ID 31337?

It is Hardhat's local chain ID. Wallets include it in signatures so transactions cannot be replayed on other networks, and the dashboard disables writes if MetaMask is on a different chain.

16. Explain signing, confirmation and reading.

Signing: MetaMask signs with the private key, proving the sender, but nothing is on-chain yet. Confirmation: the node includes the transaction in a block and the receipt shows success. Reading: an `eth_call` returns data with no signature, fee or block.

17. Why can customers view passports without a wallet?

`getPublicPassport` is a view function with no role check; the dashboard calls it through a direct RPC connection, which needs no signature.

18. How does the QR code work?

`qrcode.react` draws a QR code for `http://localhost:5173/?passport=<encoded ID>`. Opening that URL makes the app read the `passport` parameter and call `getPublicPassport`. It only works on the same computer because it points to localhost.

19. What is the difference between the public and full passport?

The public passport returns 11 fields to anyone. The full passport, for Admin and Regulator, adds the supplier's address, registration time and refining details.

20. Is the full passport private?

No. The check limits who can call the function, but on-chain data is readable at node level, and the `from` address of an `eth_call` can be set freely. It is access control, not confidentiality.

21. How did you test the contract?

Ten Solidity tests using forge-std, run with `npx hardhat test solidity`. `vm.prank` simulates different callers and `vm.expectRevert` checks exact error messages. All ten pass.

22. What do your tests not cover?

Event contents, every validation message, revokeRole, getAllBatchIds, individual wrong-role tests for later stages, and the frontend. Some UI checks are still pending.

23. What happens if you restart the Hardhat node?

The in-memory chain is erased. I redeploy (normally the same address, since account #0 deploys first), re-export and re-seed, and clear MetaMask's activity data to reset nonces.

24. What are events used for?

They write logs (RoleGranted, RoleRevoked, BatteryRegistered, StageAdvanced) into receipts as an audit trail that off-chain tools can search. Our dashboard does not display them yet.

25. Can recorded data be changed?

Not through the contract: each stage function works once per batch and there is no edit or delete function. On a real network, past blocks are also protected by consensus.

26. How does your project relate to Volvo and Circulor?

It is inspired by their publicly described battery-traceability work. It is my own educational implementation, not connected to their systems and not an official product.

13.5 Harder follow-up questions

"If blockchain can't verify truth, what's the point?"

It gives accountability and tamper-evidence: every value is tied to a responsible account and cannot be changed later. If false data is found, you know exactly who submitted it and when. Truth needs complementary controls such as audits and certificates.

"Your network has one node that you control. Isn't that just a database?"

For the prototype, yes: it demonstrates the contract logic, signing and the role model, not decentralisation. The same contract could run on a multi-party or public network, where the tamper-resistance would be real.

"Why is the Admin a single point of failure? How would you fix it?"

One key controls all roles. A fix is multi-signature approval or several admins using an established access-control library, plus secure key storage.

"What stops the Admin from making themselves a Supplier and registering fake batches?"

Nothing in the contract. The Admin can grant the Supplier role to another address it controls. (Granting it to its own address would *replace* its Admin role, because each address holds exactly one role.) The audit trail, `RoleGranted` events and `registeredBy`, would reveal who did it. Production systems limit a single admin with governance such as multi-signature approval.

"Why store full text on-chain rather than hashes?"

For a transparent, easy-to-demo prototype. In production, sensitive or large data would live off-chain with only hashes stored on-chain, which is cheaper and more private.

"Why does refining an unregistered batch give 'Battery not found' rather than 'Stage order violated'?"

Modifiers run in order: `onlySupplier`, then `batteryExists`, then `atStage`. The existence check fails first, which gives a clearer message. Test 6 checks exactly this.

"What is the purpose of simulateContract if the contract checks everything anyway?"

User experience: it shows the contract's own error before the user signs a transaction that would fail. Security still comes from the contract when the real transaction executes.

"Why is the deployed address always 0x5FbD...?"

A contract address is derived from the deployer's address and nonce. On a fresh node, account #0's first transaction is always the deployment, so the address is the same each time.

"How would the QR code work for real customers?"

Host the dashboard on a public HTTPS URL and point it at a reachable network (testnet, public or permissioned). The QR code would then encode that public URL. This is future work.

13.6 Statements to avoid, and what to say instead

Avoid	Say instead
"The blockchain guarantees the data is true."	"It guarantees the record has not changed and shows who submitted it."
"The full passport is private."	"It is access-restricted at the function level; on-chain data is not confidential."
"It's deployed on Ethereum."	"It runs on a local Hardhat development network, chain ID 31337."
"This is Volvo's system." / "It integrates with Circulor."	"It is an educational prototype inspired by their case."
"It's decentralised."	"The contract is designed for a shared network; my demo runs on a single local node."
"The UI controls who can do what."	"The contract enforces permissions; the UI only shows the forms for your on-chain role."
"Everything is fully tested."	"Ten tests cover the core rules; some validation, events and UI flows are not automated."
"All six stages were completed for DEMO-BATCH-0002."	Say this only after you finish Chapter 11; until then: "DEMO-BATCH-0002 has reached Refined."
"The tests use MetaMask."	"The tests use Hardhat's test EVM with simulated callers (`vm.prank`)."
"Gas is free."	"On my local chain gas is paid with worthless test ETH; real networks charge real fees."

13.7 Glossary

Term	Meaning
ABI	Application Binary Interface: the list of a contract's functions and events, used to encode calls and decode results.
Address	A 20-byte account identifier (0x…), derived from a public key; also identifies contracts.
Artifact	Hardhat's compiler output file (ABI, bytecode) in `artifacts/`.
Automine	Hardhat mode that mines a new block for every transaction immediately.
Block	A batch of transactions linked to the previous block by its hash.
Bytecode	The compiled program the EVM executes.
Chain ID	Number identifying a network (31337 for Hardhat local).
Cheatcode	Test-only command such as `vm.prank` or `vm.expectRevert`.
Component (React)	A function that returns part of the user interface.
Consensus	How independent nodes agree on new blocks (not present on our single node).
Constructor	Contract code that runs once at deployment.
Contract address	Where a deployed contract lives; derived from deployer address and nonce.

Term	Meaning
Decentralisation	No single party controls the system.
Deploy	Send a transaction that creates a contract on a network.
Digital signature	Proof, made with a private key, that an account approved a transaction.
Enum	A fixed list of named values stored as numbers (Role, Stage).
eth_call	JSON-RPC read request; executes a function without a transaction.
Event / log	A record emitted by a contract into the transaction receipt.
EVM	Ethereum Virtual Machine, which runs contract bytecode.
forge-std	Foundry's Solidity testing library, used by our tests.
Gas	Unit of computational work; fee = gas × gas price.
Hardhat	Ethereum development environment: compiler, tests, local node, scripts.
Hash	A one-way fixed-length fingerprint of data (Keccak-256 on Ethereum).
Immutability	Recorded data cannot be changed afterwards.
Indexed (event field)	Event field stored as a searchable topic; strings are stored as their hash.
JSON-RPC	Request/response protocol nodes use, with messages written in JSON.
Mapping	Solidity key → value table where unset keys return a default value.
MetaMask	Browser wallet that stores keys and signs transactions.
Modifier	Reusable check that runs before a function body.
msg.sender	The address calling the current function.
Node	A computer running blockchain software and holding the chain data.
Nonce	Per-account transaction counter preventing replay and ordering transactions.
npm / npx	Node's package manager / its command for running package tools.
Oracle problem	The difficulty of getting trustworthy real-world facts onto a blockchain.
Permissioned blockchain	A network where only approved parties run nodes or read data.
Private / public key	Secret signing key / key derived from it, from which the address comes.
Props / state	React inputs to a component / values a component remembers.
Receipt	Node's record of a mined transaction: block, status, gas, logs.
Revert	Failure that undoes all changes of a call and returns an error.
RPC endpoint	URL where a node accepts requests (`http://127.0.0.1:8545`).
Seed (script)	Our script that grants roles and creates DEMO-BATCH-0001.
Selector	First 4 bytes of the hash of a function signature; identifies the function in calldata.
simulateContract	viem function that dry-runs a write with `eth_call` to catch errors.
Smart contract	Program deployed on a blockchain.

Term	Meaning
Solidity	The language our contract is written in (version 0.8.34).
State machine	System with fixed states and allowed transitions (our lifecycle).
Struct	A group of named fields (Battery, PublicPassport).
Transaction	A signed request that changes blockchain state.
TypeScript	JavaScript with types; used for scripts and the frontend.
viem	TypeScript library for interacting with EVM chains.
View function	Function that only reads state; callable for free.
Vite	Development server and build tool for the dashboard.
Wallet	Software holding private keys and signing transactions.

APPENDIX

Project progress checklist

The complete status of the project in one table.

Item	Status	Evidence
Contract written (roles, 6 stages, validation, events, reads)	VERIFIED	`contracts/BatteryPassport.sol`
Contract compiles (Solidity 0.8.34)	VERIFIED	Compile output
10 Solidity tests pass	VERIFIED	"10 passing"
Deployed to local node	VERIFIED	0x5fbd...0aa3, block 1
Roles granted to accounts #1–#4	VERIFIED	Seed output; direct contract read
DEMO-BATCH-0001 through all six stages	VERIFIED	"Final stage: Recycled (6)"
ABI and address exported to frontend	VERIFIED	"17 entries"
Frontend production build	VERIFIED	`npm run build`
Public passport lookup and QR code	VERIFIED	Your observation
MetaMask on Hardhat Local; Supplier role displayed	VERIFIED	Your observation
DEMO-BATCH-0002 registered via dashboard	VERIFIED	Block 12
DEMO-BATCH-0002 refined via dashboard	VERIFIED	Tracker at Refined
Manufacturing and distribution via dashboard (Manufacturer)	PENDING	Chapter 11.1 A–B
End of life and recycling via dashboard (Supplier)	PENDING	Chapter 11.1 C–D
Regulator audit panel (full passport, batch list)	NOT YET VERIFIED IN UI	Contract-level read checked; UI pending
Customer view; Admin grant/revoke in UI	NOT YET VERIFIED IN UI	Chapter 11.2
Invalid-action checks in UI	PENDING	Chapter 11.3
Screenshots; update presentation results	PENDING	Chapter 11.5
Permissioned network, off-chain data, audits, multisig admin, public hosting	FUTURE WORK	Chapter 12.5

One-page revision sheet

The project in one line

A local-blockchain prototype of an EV battery passport: one Solidity contract enforces 5 roles and a 6-stage lifecycle; a React dashboard shows a public passport (no wallet) with a QR link and lets authorised accounts record stages via MetaMask.

Stack

- Solidity 0.8.34 · Hardhat 3.20.0 · chain ID 31337 · RPC 127.0.0.1:8545
- React 19 + TypeScript + Vite 8 · viem 2.57 · MetaMask · qrcode.react 4.2 · forge-std 1.17

Roles (enum Role)

- 0 None · 1 Admin (grant/revoke; audit) · 2 Supplier (register, refine, end of life, recycle) · 3 Manufacturer (manufacture, distribute) · 4 Regulator (full passport, batch list) · 5 Customer (public view only)
- Deployer = Admin (constructor). Role checked on `msg.sender`.

Lifecycle (enum Stage)

- 0 Unregistered → 1 Registered → 2 Refined → 3 Manufactured → 4 Distributed → 5 EndOfLife → 6 Recycled
- Each step: one function, one role, one allowed starting stage (`atStage`). No skipping, repeating or going back.

Key error messages

- Access denied: <Role> role required
- Battery not found: batch ID does not exist
- Stage order violated: prerequisite stage not yet complete
- Duplicate: a battery with this batch ID already exists

Reads vs writes

- Read = `eth_call` : free, no signature, no block (public passport, getRole).
- Write = signed transaction → block → receipt (status success).

Write path

Form → `simulateContract` (dry run as you) → `writeContract` → MetaMask signs → node executes + mines → `waitForTransactionReceipt` → "Confirmed in block N" → re-read passport.

Demo commands

- T1: `npx hardhat node`
- T2: `npx hardhat run scripts/deploy.ts --network localhost` · `…/export-frontend.ts` · `…/seed.ts --network localhost`
- T3: `cd frontend` · `npm run dev` → localhost:5173
- Tests: `npx hardhat test solidity` → 10 passing

Accounts

- #0 Admin · #1 Supplier · #2 Manufacturer · #3 Regulator · #4 Customer (public test keys: never real funds)

Architecture

- Write: React → viem (simulate, encode) → MetaMask (approve, sign, send) → Hardhat node → BatteryPassport.sol
- Read: React → viem public client → `eth_call` at 127.0.0.1:8545 (no wallet)
- Build: compile → deploy.ts (address) → seed.ts → export-frontend.ts → contract.ts

Top limitations

- Local single node; state lost on restart; not decentralised.
- Data truth not verified; demo data fictional.
- On-chain data is public; full-passport check is not privacy.
- Single Admin key; no corrections; QR works only on this computer.

Status

- **Done:** compile, 10 tests, deploy, seed (0001 → Recycled), export, build, public lookup + QR, MetaMask + Supplier role, 0002 registered (block 12) → Refined.
- **Pending:** 0002 manufacture → distribute (#2), end of life → recycle (#1); Regulator, Customer and Admin views; UI negative tests; screenshots.

Three sentences to remember

- "Blockchain guarantees the integrity of the record, not the truth of its content."
- "Access control decides who can change data; it does not make data private."
- "It's an educational prototype on a local Hardhat chain, inspired by, not connected to, Volvo and Circulor."